

UNIVERSIDAD DE MÁLAGA

TRABAJO FIN DE GRADO



PABLO GARCÍA RAMÍREZ, MÁLAGA, 2025.

Agradecimientos

Este proyecto debe su agradecimiento a mi tutor Daniel Garrido Márquez, Cotutor Cristian Martín Fernández, además de todos los profesores que han formado parte de mi formación como ingeniero telemático. Por su parte, también quiero agradecer el apoyo incondicional de mis familiares y seres queridos tanto a lo largo de mi carrera estudiantil, como en la elaboración de este trabajo final.

Lista de Acrónimos

En esta lista se hace un resumen de los acrónimos existentes en la memoria, indicando su significado en orden cronológico de aparición. Para tecnicismos y abreviaciones inglesas, se ha utilizado su directa traducción al español.

ETSIT	Escuela Técnica Superior de Ingeniería de Telecomunicación
IA	Inteligencia Artificial
IU	Interfaz de Usuario
App	Aplicación
IDE	Entorno de Desarrollo Integrado
SDK	Kit de Desarrollo de Software
JIT	Justo a Tiempo
AOT	Antes de Tiempo
API	Interfaz de Programación de Aplicaciones
RPC	Llamada a Procedimiento Remoto
JSON	Notación de Objeto de JavaScript
NPM	Gestor de Paquetes de Nodos
HTTP	Protocolo de Transferencia de Hipertexto
CU	Caso de Uso
CRUD	Crear, Leer, Actualizar, Eliminar
CORS	Compartir Recursos entre Orígenes Cruzados
URL	Localizador de Recursos Uniforme

**APROXIMACIÓN DE DIAGNÓSTICOS MÉDICOS BASADOS EN
INTELIGENCIA ARTIFICIAL**

Autor: Pablo García Ramírez

Tutor: Daniel Garrido Márquez

Cotutor: Cristian Martín Fernández

Departamento: Lenguajes y ciencias de la computación

Titulación: Grado en Ingeniería Telemática

Resumen

Impulsados por los diversos avances tecnológicos presentes hoy en día en nuestro entorno y en el desarrollo de distintas herramientas que poco a poco empiezan a componer nuestra vida cotidiana, este trabajo pretende esbozar un prototipo que ofrezca un marco sólido para la eficiencia en la salud pública desde ahora en adelante en tantos hospitales como sea posible.

La Inteligencia Artificial es un poderoso recurso actualmente presente en gran variedad de ámbitos y aplicaciones. A consecuencia de esto, en este trabajo se pretende aprovechar esta capacidad de aprendizaje vertiginosa por parte de los ordenadores, para alcanzar a mejorar la gestión de pacientes en consultas médicas y aumentar la fiabilidad y rapidez de las mismas.

Como resultado de este Trabajo Fin de Grado se ha desarrollado una aplicación web polivalente, que permite controlar una base de datos global de pacientes y médicos, además de un registro de informes médicos y diagnósticos asignados con ayuda de la IA.

El sistema propuesto se ha llevado a cabo a través de tecnologías como Flutter o Firebase, previamente diseñadas por Google.

**MEDICAL DIAGNOSIS APPROACHING BASED ON ARTIFICIAL
INTELLIGENCE**

Author: Pablo García Ramírez

Tutor: Daniel Garrido Márquez

Co-Tutor: Cristian Martín Fernández

Departament: Languages and Computer Science

Titulation: Degree in Telematics Engineering

Abstract

Driven by the various technological advances present in our environment today and the development of different tools that are gradually beginning to make up our daily lives, this work aims to outline a prototype that offers a solid framework for efficiency in public health from now on in as many hospitals as possible.

The Artificial Intelligence is a currently powerful feature present in a wide range of fields and applications. As a result, this work tries to leverage this rapid machine learning capacity, to reach user – management optimization and rise the reliability and speed in medical consultations.

As a result of this final degree project, it has been implemented a web application, which allows to control a globalised data base, as well as AI aided medical reports and assigned diagnoses.

This proposed system has been carried out through different technologies such as Flutter or Firebase, designed by Google.

Contenido

Capítulo 1. Introducción	13
1.1 Motivación	13
1.2 Objetivos	13
1.3 Estructura de la memoria.....	14
Capítulo 2. Estado del Arte.....	15
2.1 Inteligencia Artificial en Medicina	15
2.1.1 ¿Qué es la Inteligencia Artificial en medicina?	15
2.1.2 ¿Cómo se usa la IA en medicina?	15
2.1.3 Beneficios de la IA en medicina.....	15
2.2 Tecnologías de reconocimiento por voz	16
2.3 Aplicaciones similares existentes	16
2.3.1 Desarrollo acelerado de fármacos.....	16
2.3.2 Análisis de imágenes médicas	16
Capítulo 3. Metodología	17
3.1 Planificación y fases del desarrollo	17
3.2 Metodologías de desarrollo	17
3.2.1 Scrum	18
3.2.2 Kanban	18
Capítulo 4. Tecnologías utilizadas.....	19
4.1 Flutter.....	19
4.1.1 Dart	20
4.1.2 Widgets.....	20
4.1.3 Estructura del framework	20
4.1.4 Hot Reload y Hot Restart	20
4.2 Firebase	21
4.2.1 Servicios Firebase utilizados en el proyecto	21
4.2.2 Firebase Authentication	21
4.2.3 Firestore Database.....	21
4.3 Backend y Firebase Functions	23
4.3.1 Node.js.....	23
4.3.2 Firebase Functions	24
4.3.3 Hugging Face	24
4.4 Herramientas de desarrollo	26

4.4.1 Postman	26
4.4.3 Git/GitHub	27
4.5 Otros	27
4.5.1 JSON	27
4.5.2 NPM	28
4.5.3 HTTP/HTTPS	28
4.5.4 Draw.io	29
4.5.5 PlantUML	29
4.5.6 Mermaid Live Editor	29
4.5.7 Canva	30
Capítulo 5. Análisis del Sistema	31
5.1 Descripción general	31
5.2 Requisitos funcionales	32
5.3 Requisitos no funcionales	33
Capítulo 6. Especificación del Sistema	35
6.1 Casos de Uso	35
6.2 Flujo de Información entre los módulos	40
Capítulo 7. Diseño del Sistema	43
7.1 Arquitectura General	43
7.2 Diseño de la Base de Datos	44
7.3 Estructura del módulo Backend	46
7.4 Estructura del módulo Frontend	47
7.4.1 Arquitectura interna del Frontend	47
7.4.2 Estructura de Carpetas y Organización del Código	49
7.4.3 Navegación	52
Capítulo 8. Implementación	55
8.1 Gestión y desarrollo interno	55
8.1.1 Módulo de gestión de usuarios	55
8.1.2 Carga y análisis de documentos	58
8.1.3 Conversión de voz a texto	62
8.2 Decoración	64
8.2.1 Principios de diseño	65
8.2.2 Estructura General de los Widgets	65
Capítulo 9. Pruebas y Validación	67

9.1 Pruebas funcionales.....	67
9.1.1 Autenticación del usuario.....	67
9.1.2 Gestión CRUD de Médicos y Pacientes.....	68
9.1.3 Acceso a enlaces externos	68
9.1.4 Análisis de informes	68
9.1.5 Transcripción de voz a texto.....	69
9.2 Pruebas de integración	69
9.3 Posibles mejoras detectadas.....	71
Capítulo 10. Conclusiones y Trabajo Futuro	75
10.1 Resumen de los logros alcanzados	75
10.2 Limitaciones y mejoras para futuras versiones	76
Capítulo 11. Manual de Instalación.....	79
11.1 Requerimientos	79
11.2 Pasos de instalación.....	79
Capítulo 12. Manual de Usuario	83
12.1 Inicio de Sesión	83
12.2 Página de restablecimiento de contraseña	84
12.3 Home Page	86
12.4 Páginas de gestión de personas	88
12.5 Página de análisis de informes médicos	92
12.6 Página de reconocimiento de voz y análisis	96
Capítulo 13. Referencias bibliográficas	103
13.1 Fuentes utilizadas, estándares y documentación técnica.....	103

Figura 1- Scrum logo	18
Figura 2- Ejemplo tablero Kanban	18
Figura 3- Flutter logo	19
Figura 4- Dart logo	20
Figura 5- Firebase logo.....	21
Figura 6- Node.js logo.....	23
Figura 7- Hugging Face logo	25
Figura 8- Postman logo	26
Figura 9- Visual Studio Code logo.....	27
Figura 10- Git logo.....	27
Figura 11- Logo Draw.io	29
Figura 12- Logo PlantUML.....	29
Figura 13- Mermaid Live Editor logo	29
Figura 14- Canva logo	30
Figura 15- Diagrama de secuencia para la autenticación de usuario	40
Figura 16- Diagrama de secuencia para el procesamiento de documentos...	41
Figura 17- Diagrama de secuencia de conversión de voz.....	41
Figura 18- Diagrama de secuencia para visualización de diagnósticos.....	42
Figura 19- Esquema general del diseño del sistema	43
Figura 20- Diagrama entidad-relación.....	44
Figura 21- Ejemplo de colecciones reales en Firestore database	45
Figura 22- Ejemplo real de Firebase Functions	46
Figura 23- Carpeta de almacenamiento de Firebase Storage	46
Figura 24- Documento almacenado en la carpeta tras ser analizado	47
Figura 25- Capa de Presentación	47
Figura 26- Capa de Gestión de Estado	48
Figura 27- Capa de Servicio.....	48
Figura 28- Carpeta 'assets' Flutter project.....	49
Figura 29- Carpeta 'functions' Flutter project	49
Figura 30- Carpeta 'pages' Flutter project	50
Figura 31- Carpeta 'services' Flutter project	50
Figura 32- Scripts restantes Flutter project.....	50
Figura 34- Diagrama de navegación entre ventanas.....	53
Figura 35- Código: Función para inicio de sesión	55
Figura 36- Código: Función para reestablecer contraseña	56
Figura 37- Código: Función para leer pacientes en tiempo real	57
Figura 38- Código: Función para añadir pacientes	57
Figura 39- Código: Función para actualizar pacientes	57
Figura 40- Código: Función para eliminar pacientes	58
Figura 41- Código: Función para la subida de archivos	58
Figura 42- Código: Función para análisis de documentos	59
Figura 43- Código: Importación de módulos en index.js	60
Figura 44- Código: Configuración del servidor en index.js	60
Figura 45- Código: Creación de la función en index.js	60
Figura 46- Código: Instrucción a enviar desde index.js	61
Figura 47- Código: Petición a HuggingFace en index.js	61

Figura 48- Código: Procesamiento de la respuesta en index.js	61
Figura 49- Código: Control de errores en index.js	62
Figura 50- Código: Librerías necesarias en Voz-Texto	62
Figura 51- Código: Definición de clase y variables Voz-Texto	62
Figura 52- Código: Función para alternar escucha Voz-Texto	63
Figura 53- Código: Función de escucha Voz-Texto	63
Figura 54- Código: Detención y Reset Voz-Texto	63
Figura 55- Código: Envío a IA Voz-Texto	64
Figura 56- Código: Estructura de diseño geneneralizada	65
Figura 57- Pasos de instalación: 'flutter doctor'	80
Figura 58- Pasos instalación: 'Create new token'	81
Figura 59- Pasos de instalacion: Permiso de 'Inference Providers'	81
Figura 60- Aplicación: Página de Log In	83
Figura 61- Aplicación: Ejemplo para credenciales incorrectas	83
Figura 62- Aplicación: Página de restablecimiento de contraseña	84
Figura 63- Aplicación: Botón de retroceso a la página de autenticación	84
Figura 64- Aplicación: Mensaje de correo enviado	84
Figura 65- Aplicación: Correo para la recuperación de contraseña	85
Figura 66- Aplicación: Interfaz de reset	85
Figura 67- Aplicación: Home Page	86
Figura 68- Aplicación: Web alternativa	86
Figura 69- Aplicación: Botón para cierre de sesión	87
Figura 70- Aplicación: Mensaje de cierre de sesión	87
Figura 71- Aplicación: Ventanas de gestión de pacientes y médicos	88
Figura 72- Aplicación: Botón de creación de personas	89
Figura 73- Aplicación: Ventanas de creación de personas	89
Figura 74- Aplicación: Icono de eliminación de personas	90
Figura 75- Aplicación: Inicio de la edición de un paciente	91
Figura 76- Aplicación: Final de la edición de un paciente	91
Figura 77- Aplicación: Ventana de Informes Médicos	92
Figura 78- Aplicación: Botón para la carga de archivos	92
Figura 79- Explorador de archivos	93
Figura 80- Aplicación: Mensaje de éxito en la carga de archivo	93
Figura 81- Aplicación: Mensajes indicativos por la terminal	94
Figura 82- Aplicación: Diagnóstico mostrado en la IU	94
Figura 83- Aplicación: Botón para visualizar historial de archivos	94
Figura 84- Aplicación: Historial de archivos	95
Figura 85- Aplicación: Contenido de archivos	95
Figura 86- Aplicación: Botón para visualizar ocultar de archivos	96
Figura 87- Aplicación: Ventana de selección de pacientes	96
Figura 88- Aplicación: Desplegable de selección de pacientes	96
Figura 89- Aplicación: Botón de continuación	97
Figura 90- Aplicación: Ventana de diagnóstico personalizado	97
Figura 91- Aplicación: Botón de retroceso en la selección de pacientes	98
Figura 92- Aplicación: Botón de acción del micrófono	98
Figura 93- Aplicación: Permisos de uso de micrófono	98

Figura 94- Aplicación: Botón de STOP del micrófono	99
Figura 95- Aplicación: Transcripción de voz exitosa	99
Figura 96- Aplicación: Botón de envío de texto médico hacia IA	99
Figura 97- Aplicación: Terminal de ejecución del reconocimiento de voz ...	100
Figura 98- Aplicación: Respuesta aplicada al chat médico	100
Figura 99- Base de datos: Colecciones divididas en los pacientes con historial médico.....	101
Figura 100- Base de datos: Registro de la conversación almacenado.....	101

Capítulo 1. Introducción

1.1 Motivación

La sanidad es un recurso necesario y requerido por todos, pero no siempre es tan resolutivo o eficaz como espera ser. La atención médica es imprescindible a lo largo de nuestras vidas y se invierte mucho capital en mejorar infraestructuras hospitalarias, calidad del servicio y técnicas innovadoras contra enfermedades peligrosas.

A pesar de esto, en ocasiones los avances no llegan con suficiente rapidez y, aprovechando la revolución tecnológica que estamos presenciando, los robots y programas informáticos cada vez son más frecuentes en estos entornos.

Este proyecto se enfocará en añadir mejoras sanitarias utilizando la Inteligencia Artificial, una herramienta fundamental hoy en día, que está mejorando multitud de campos en nuestra sociedad y que debemos aprovechar al máximo para sacar partido de sus propiedades.

1.2 Objetivos

El objetivo principal de este proyecto es desarrollar una aplicación web que utilice Inteligencia Artificial para asistir consultas médicas, facilitando la gestión de información y la comunicación entre médicos y pacientes. A consecuencia de esto, se proponen los siguientes objetivos específicos a cumplir:

1. **Desarrollar una plataforma accesible**, eficiente y con una IU sencilla para todos, que facilite estas gestiones de forma estructurada.
2. **Controlar y gestionar individuos**, para tener un historial de registro accesible para todos los datos involucrados.
3. **Integrar en la App el uso de Inteligencia Artificial**, para utilidades avanzadas como el análisis de documentos.
4. **Implementar reconocimiento de voz**, para agilizar el proceso de redacción y envío de datos.
5. **Optimizar el flujo de trabajo del personal sanitario**, reduciendo tiempos de espera y mejorando la fiabilidad de los diagnósticos.
6. **Garantizar un sistema seguro y fiable**, usando plataformas con Firebase para un almacenamiento ordenado e integración de autenticación.

1.3 Estructura de la memoria

Aquí se desglosan los diferentes capítulos y apartados que componen en el proyecto, de forma resumida, y se exponen los fundamentos de cada punto:

1. Introducción: En este primer apartado se expresa la motivación del proyecto, los objetivos que se esperan cumplir con la realización de este, y la organización con la que se divide.
2. Estado del Arte: Aquí se realiza una vista general a cómo de integrada está la IA en distintas aplicaciones, breve explicación de tecnologías de reconocimiento de texto y/o audio y aplicaciones similares existentes.
3. Metodología: Explicación de la metodología utilizada en la elaboración del proyecto.
4. Tecnologías utilizadas: Se resaltarán los distintos recursos empleados, así como lenguajes de programación, IDE, SDK...
5. Análisis del sistema: Corresponde a los requisitos funcionales y no funcionales, efectuando una valoración general.
6. Especificación del sistema: Casos de uso e información de los requisitos extraídos del apartado anterior.
7. Diseño del sistema: Describe la arquitectura y el diseño de la aplicación, abordando la estructura de la base de datos, el flujo de información y otros aspectos relacionados con la implementación técnica.
8. Implementación: Aquí se documenta el proceso de desarrollo de la aplicación web.
9. Pruebas y validación: Se detallan las comprobaciones realizadas para valorar el funcionamiento del sistema.
10. Conclusiones: En este capítulo se reconocerán los logros alcanzados y objetivos cumplidos, además se tendrán en cuenta futuras mejoras.
11. Guía de Instalación: Ofrece una breve guía sobre cómo empezar este proyecto, con los recursos a utilizar e instalaciones requeridas.
12. Manual de Usuario: Se exponen de forma ilustrativa los elementos que compone la aplicación y una explicación del uso de sus funciones.

13. Referencias: En este capítulo final se exponen las diferentes fuentes de información y documentación empleadas y requeridas.

Capítulo 2. Estado del Arte

2.1 Inteligencia Artificial en Medicina

2.1.1 ¿Qué es la Inteligencia Artificial en medicina?

[1] La inteligencia artificial en medicina es el uso de modelos combinados con machine-learning para ayudar a procesar datos médicos y brindar a los profesionales médicos información importante, mejorando los resultados de salud y las experiencias de los pacientes.

Su práctica se basa en el uso de algoritmos que favorecen el aprendizaje automático, esto genera una serie de beneficios como: análisis de datos médicos, predicción de enfermedades, optimización de tratamientos o ayuda en diagnósticos.

2.1.2 ¿Cómo se usa la IA en medicina?

Actualmente, las funciones más comunes de la IA en entornos médicos son el apoyo a la toma de decisiones clínicas y el análisis de imágenes. Las herramientas de apoyo a la toma de decisiones clínicas ayudan a los proveedores a tomar decisiones sobre tratamientos, medicamentos, salud mental y otras necesidades de los pacientes.

Esta herramienta está en período de prueba constante antes de pasar a una implementación segura dentro de la sanidad. La investigación y los resultados de estas pruebas aún se están recopilando y aún se están definiendo los estándares generales para el uso de la IA en medicina.

2.1.3 Beneficios de la IA en medicina.

1. Atención informada al paciente: detalles concretos de las patologías.
2. Reducción de errores: evitando las negligencias humanas.
3. Reducción de costes de atención: no hay personal, no hay gastos.
4. Aumento del compromiso médico-paciente: mayor confianza y calidad.
5. Relevancia contextual: valorando situaciones y resultados previos.

2.2 Tecnologías de reconocimiento por voz

[2] La tecnología de reconocimiento de voz hace que el procesador sea capaz de descifrar la información que contiene la voz humana e incluso de predecir lo que el humano quiere decir para reducir la tasa de error en la transcripción. El reconocimiento automático del habla consta de dos procesos de aprendizaje diferenciados:

Por un lado, un aprendizaje deductivo que consiste en la transferencia de conocimientos del hombre a un sistema informático;

Y por otro lado un aprendizaje inductivo, aquí se trata de que el sistema sea capaz de obtener esos conocimientos a través de ejemplos.

En la ejecución de este proyecto se llevará a cabo el uso de este tipo de tecnologías que facilitarán la labor del profesional de la salud. Al estar habilitada la función de escucha por micrófono, las palabras pronunciadas por el paciente serán transcritas a texto, posiblemente editable para no cometer errores, y que posteriormente serán enviadas para el análisis por parte del modelo alojado de inteligencia artificial.

2.3 Aplicaciones similares existentes en el sector sanitario

2.3.1 Desarrollo acelerado de fármacos

[3] La IA podría ayudar a reducir los costes de desarrollo de nuevos medicamentos principalmente de dos maneras: creando mejores diseños de fármacos y encontrando nuevas combinaciones de fármacos prometedoras. Con la IA, se podrían superar muchos de los retos de 'big data' a los que se enfrenta el sector de las ciencias de la vida.

Un ejemplo concreto es Insilico Medicine, una empresa que usa IA para descubrir y diseñar moléculas potenciales para nuevos medicamentos. Su modelo utiliza aprendizaje profundo 'deep learning' para analizar enormes bases de datos de compuestos químicos y datos biológicos, identificar patrones y proponer nuevas moléculas que tengan alta probabilidad de éxito.

2.3.2 Análisis de imágenes médicas

[4] Las investigaciones han indicado que la IA impulsada por redes neuronales puede ser tan eficaz como los radiólogos humanos a la hora de detectar signos de cáncer y otras afecciones. Por otra parte, el uso de esta poderosa capacidad de computación alivia a los expertos médicos en cuanto a la cantidad de imágenes que deben ser revisadas.

Capítulo 3. Metodología

3.1 Planificación y fases del desarrollo

Para llevar a cabo todo este proceso de documentación del proyecto ha sido necesario el uso de metodología, que permiten tener una visión clara y organizada de las fases que lo componen. A continuación, se describen las etapas del desarrollo:

1. Definición y recopilación de requisitos: En esta fase inicial se llevó a cabo un análisis del potencial problema que se pretendía resolver, así como las principales funcionalidades que exigía tener el sistema.
2. Investigación y selección de tecnologías: Estudio previo comparativo de las distintas herramientas posibles a utilizar para abordar los objetivos planteados previamente.
3. Desarrollo iterativo de la aplicación: Se implementaron las funcionalidades de manera progresiva, comenzando con los módulos esenciales y evolucionando hacia una versión más completa de la aplicación.
4. Pruebas y validación: Se realizaron pruebas para la validación y verificación del sistema planteado, es decir, que funcionaba como se esperaba para distintas situaciones adversas y que lo hacía libre de errores.
5. Documentación: En este caso la parte de documentación y desarrollo de la memoria se realizó de forma simultánea a la parte práctica de implementación. Me pareció oportuno redactar a la vez que evolucionaba mi aplicación para no perder detalle del proceso y construir un proyecto sólido.

3.2 Metodologías de desarrollo

La programación de aplicaciones software no contempla un escenario de desarrollo lineal. La metodología ágil es un conjunto de técnicas aplicadas en ciclos de trabajo cortos, con el objetivo de que el proceso de entrega de un proyecto sea más eficiente.

Para garantizar una evolución adecuada se ha optado por el uso de una combinación de tipos de metodologías ágiles: Scrum y Kanban.

Gracias a las propiedades de estas metodologías ágiles, la organización del trabajo fomentó un desarrollo del proyecto óptimo.

3.2.1 Scrum

Scrum es una metodología ágil ampliamente utilizada en el desarrollo de software, caracterizada por dividir el proyecto en ciclos cortos de trabajo llamados 'sprints'. En este proyecto, se han aplicado los siguientes principios de Scrum:

- Backlog del producto: Se creó una lista de tareas y funcionalidades que debían ser implementadas.
- Desarrollo iterativo: Se implementaron las funcionalidades en ciclos cortos, permitiendo realizar ajustes en caso de ser necesario.



Figura 1- Scrum logo

3.2.2 Kanban

Además de Scrum, se utilizó Kanban para visualizar y gestionar las tareas de manera más eficiente. Kanban permite una organización flexible del trabajo, optimizando el flujo de desarrollo a través de una tabla.

Se ilustraron en dicha tabla las tareas recopiladas de la sección anterior para poder trabajar sobre ellas de una manera estructurada.



Figura 2- Ejemplo tablero Kanban

Capítulo 4. Tecnologías utilizadas

Para llevar a cabo la implementación práctica real de la aplicación web sobre la que se basa esta documentación, ha sido necesario el uso de ciertas tecnologías previamente existentes con las cuáles trabajar para lograr diseñar lo planteado.

Un proyecto como este exige variedad de conocimientos, programas informáticos, entornos de desarrollo, documentación externa, herramientas de conexión...

Esta aplicación web, y la mayoría, se construyen en base a capas, es decir, el trabajo comienza desde cero y se van introduciendo funcionalidades a medida que se van consolidando ciertos puntos de control. Cuando la capa sobre la que se está trabajando funciona y es sólida, se continúa añadiendo utilidades que posteriormente serán conectadas al sistema global.

A continuación, se exponen los diferentes recursos empleados y la utilidad de cada uno de ellos.

4.1 Flutter

[6] Flutter se corresponde con la parte FrontEnd de la aplicación. Básicamente es la parte de la aplicación que interactúa con el usuario, conocida como el lado del cliente. Los tipos de letra, colores, navegación, movimientos, enlaces, efectos del teclado y ratón, son, entre otros, distintos elementos de esta parte visual.

Se trata de un marco de código abierto diseñado por Google para crear aplicaciones multiplataforma compiladas de forma nativa a partir de una única base de código. Transforma el proceso de desarrollo y permite crear, probar e implementar experiencias atractivas para dispositivos móviles, web, de escritorio e integrados desde una única base de código.

Esta capa del proyecto será la encargada de brindar una experiencia atractiva al usuario, con una interfaz de fácil manejo y combinando sus distintos elementos para lograr un conjunto interactivo y funcional.



Figura 3- Flutter logo

4.1.1 Dart

Dart es el lenguaje de programación optimizado que utiliza Flutter para las aplicaciones multiplataforma. Estas son algunas de sus características principales:

- Orientado a objetos y con una sintaxis similar a Java
- Compilación JIT y AOT para mejorar el rendimiento
- Manejo sencillo de asincronía
- Tipado fuerte y flexible que permite seguridad en el código



Figura 4- Dart logo

4.1.2 Widgets

En este entorno de programación, todo es un widget. Los widgets son componentes reutilizables de código que definen la apariencia y la distribución visual del interfaz. Existen distintos tipos de widgets:

- Widgets sin estado, no cambian al ser creados
- Widgets con estado, pueden cambiar su apariencia en tiempo de ejecución
- Widgets de disposición, permiten estructurar la interfaz

4.1.3 Estructura del framework

Flutter se divide en capas que controlan diferentes secciones del código. Las principales son:

1. Framework: Incluye los widgets y herramientas de la interfaz de usuario.
2. Motor renderizado: Encargado de dibujar en la pantalla los elementos.
3. Capa de integración: Conecta Flutter con las APIs nativas de plataforma.

4.1.4 Hot Reload y Hot Restart

Flutter permite acelerar el desarrollo mediante 'Hot Reload', una funcionalidad que actualiza la UI en tiempo real sin perder el estado de la aplicación. 'Hot Restart', en cambio, recompila la app desde cero, útil cuando hay cambios más profundos en el código.

4.2 Firebase

[7] Firebase es otra plataforma desarrollada por Google que proporciona la infraestructura de nube para el desarrollo de aplicaciones web y móviles. Esta herramienta permite la gestión de bases de datos, autenticación de usuarios, almacenamiento de archivos y otras funcionalidades.



Figura 5- Firebase logo

4.2.1 Servicios Firebase utilizados en el proyecto

Para el correcto funcionamiento de la app han sido imprescindibles los siguientes servicios de Firebase:

1. Firebase Authentication: Sistema de autenticación de usuarios.
2. Firestore Database: Base de datos NoSQL en tiempo real.
3. Firebase Storage: Almacenamiento de documentos PDF.
4. Cloud Functions: Framework de ejecución de solicitudes.

4.2.2 Firebase Authentication

Este servicio permite el inicio de sesión de cualquier usuario que intente entrar en la aplicación para comprobar si está autorizado para ello. Existen diferentes métodos para cumplir esta función, pero el utilizado en esta app es a través de correo electrónico y contraseña.

Se podrían añadir utilidades adicionales como inicio de sesión a través de cuentas de Google o gestión de sesiones y recuperación de contraseñas a través de enlaces email.

La autenticación se gestiona con Firebase SDK, facilitando la seguridad y evitando la necesidad de un backend personalizado para este propósito.

4.2.3 Firestore Database

Cloud Firestore es una base de datos NoSQL alojada en la nube a la que pueden acceder tus apps para Apple, Android y la Web directamente desde los SDK nativos. Cloud Firestore también está disponible en los SDKs nativos de Node.js, Java, Python, Unity, C++ y Go, además de las APIs de REST y RPC.

A partir del modelo de datos NoSQL de Cloud Firestore, se almacenan los datos en documentos que contienen campos que se asignan a valores.

Estos documentos se almacenan a su vez en colecciones, que son contenedores para tus documentos y que se pueden usar para organizar tus datos y compilar consultas.

También es posible crear colecciones inferiores dentro de documentos y crear estructuras de datos jerárquicas que se ajustan a escala a medida que tu base de datos crece. Además, las consultas de Cloud Firestore son expresivas, eficientes y flexibles. Es posible crear consultas superficiales para recuperar datos en el nivel del documento, sin la necesidad de recuperar la colección completa ni las subcolecciones anidadas.

Estas son sus propiedades más importantes:

- Flexibilidad
- Consultas expresivas
- Actualizaciones en tiempo real
- Asistencia sin conexión
- Diseñado para ajustarse a escala

4.2.4 Firebase Storage

Se basa en la infraestructura rápida y segura de Google Cloud para desarrolladores de apps que necesitan almacenar y entregar contenido generado por usuarios. Los SDK de Firebase para Cloud Storage agregan la seguridad de Google a las operaciones de carga y descarga de archivos de tus apps de Firebase, sin importar la calidad de la red. Funciones clave:

1. Operaciones robustas: Los SDKs de Firebase para Cloud Storage realizan las cargas y descargas sin importar la calidad de la red.
2. Seguridad sólida: Los SDKs de Firebase para Cloud Storage se integran en Firebase Authentication para proporcionar una autenticación sencilla e intuitiva para los desarrolladores.
3. Gran escalabilidad: Cloud Storage se diseñó con el fin de escalar a exabytes si tu app se vuelve viral.

4.3 Backend y Firebase Functions

En una aplicación común, el Backend se corresponde con el servicio o parte del código encargado de la gestión de datos, control de la lógica, autenticación, comunicación con servicios externos... En este proyecto no se ha desarrollado un backend tradicional con un servidor propio, sino que se ha optado por una arquitectura sin servidor o 'servless', donde Firebase Functions actúa como intermediario y conecta la aplicación Flutter con los modelos de IA alojados en Hugging Face.

Además, para ello se utiliza Node.js, un entorno de ejecución a través de JavaScript que maneja solicitudes HTTP y realiza llamadas a APIs externas. El uso de Cloud Functions facilita la integración con la web externa a la que nos pretendemos conectar sin necesidad de desplegar un servidor propio, esto proporciona escalabilidad y reduce carga del frontend.

4.3.1 Node.js

[9] Gracias a esta herramienta, se ha conseguido ejecutar un formato de servidor eficiente para manipular el flujo de entrada y salida que se lleva a cabo durante la ejecución de la aplicación. Su diseño lo hace capaz para manejar conexiones asíncronas, lo que lo hace ideal para aplicaciones que requieran realizar varias solicitudes simultáneas.

En este proyecto se ha utilizado Node.js dentro de Cloud Functions de Firebase, para procesar datos enviados de la aplicación Flutter y comunicarse con el modelo web de IA alojado en Hugging Face. En concreto, los archivos usados contienen la lógica necesaria para:

1. Recibir solicitudes HTTP de la aplicación con datos de entrada en formato JSON.
2. Enviar estos datos realizando peticiones POST HTTP a Hugging Face para el correspondiente análisis.
3. Procesar la respuesta proporcionada y esbozar los resultados a través de la terminal de la aplicación.



Figura 6- Node.js logo

4.3.2 Firebase Functions

Cloud Functions para Firebase es un framework sin servidores que te permite ejecutar automáticamente el código de backend en respuesta a los eventos que activan los eventos en segundo plano, las solicitudes HTTPS, el Admin SDK o los trabajos de Cloud Scheduler. Tu código JavaScript, TypeScript o Python se almacena en la infraestructura de Google Cloud y se ejecuta en un entorno administrado. No necesitas administrar ni escalar tus propios servidores.

Firebase Functions permite ejecutar código en la nube sin necesidad de gestionar un servidor propio, respondiendo a eventos en tiempo real. En este proyecto, su principal función es procesar las solicitudes de la aplicación Flutter y comunicarse con modelos de IA alojados en Hugging Face. Funciones clave:

1. Integra las funciones de Firebase y conecta Firebase con Google Cloud.
2. No necesita mantenimiento.
3. Protege la privacidad y seguridad de tu lógica.

4.3.3 Hugging Face

Hugging Face es una plataforma online dedicada a data-science y machine-learning. Básicamente es un repositorio web que almacena conjuntos de datos, modelos de inteligencia artificial, documentación, etc. Contiene modelos IA entrenados y otros más primitivos para que los entrenes a tu gusto para hacer cualquier tipo de operación que se te ocurra, desde generación de imágenes y/o vídeo, transcripción de textos, OCR, visualización de datos, modelos 3D y muchísimas más utilidades.

Se ha trabajado con Hugging Face para integrar uno de sus miles de modelos de IA en nuestra aplicación. La idea inicial era encontrar un modelo pre-entrenado con información médica para llevar a cabo la generación de diagnósticos. Sin embargo, la búsqueda de dicho modelo de IA se vio limitada a aquellos que ofrecieran un servicio a través de HF Inference API y no demasiado extenso en tamaño para garantizar éxito en las peticiones.

Esto se ha podido llevar a cabo gracias a una característica que sólo poseen ciertos modelos llamada Inference Providers. Esto es una característica que tienen y que permite usar sus servicios gracias a un Access Token, en este caso generación de texto.

En concreto se ha elegido el modelo: mistralai/Mistral-7B-Instruct-v0.3, una versión avanzada de la serie Mistral 7B, diseñada para mejorar la comprensión y generación de lenguaje natural. Esta familia de modelos está siendo popularmente usada por muchos usuarios de este repositorio por los siguientes motivos.

Cuenta con numerosas características como: tamaño y arquitectura óptimos para manejar tareas con eficiencia, ventana de contexto ampliada, tokenizador v3 que la precisión en la segmentación y procesamiento del texto, capacidad de llamada a funciones externas para interactuar con APIs...

Aunque Mistral-7B-Instruct-v0.3 no fue entrenado específicamente para tareas médicas, ha demostrado un rendimiento notable en este dominio:

- **Generación de notas de alta hospitalaria:** En un estudio centrado en pacientes cardíacos, Mistral-7B generó notas de alta que fueron evaluadas positivamente por expertos médicos en términos de relevancia clínica, completitud y legibilidad

- **Respuesta a preguntas clínicas:** En comparativas con otros modelos de tamaño medio, Mistral 7B superó a modelos especializados en biomedicina en tareas de preguntas y respuestas clínicas.

- **Versatilidad:** ha mostrado un rendimiento superior en tareas médicas multilingües.

Tras haber realizado pruebas de peticiones para valorar distintas respuestas recibidas de otros modelos de inteligencia artificial, las características que presenta este, junto las propiedades comentadas anteriormente sobre la capacidad de integración con nuestra aplicación han servido como motivo de uso para añadir este modelo concreto al proyecto.

Un aspecto negativo sería la limitación de su uso que, al igual que el resto de los modelos, la web restringe a los usuarios la capacidad de integración de la API a ciertas solicitudes, es decir, no son infinitas. Por otro lado, este modelo soporta algunos idiomas extra que, aunque la intención principal es trabajar en español, es una funcionalidad añadida.



Hugging Face

Figura 7- Hugging Face logo

4.4 Herramientas de desarrollo

4.4.1 Postman

[11] Postman es una herramienta de colaboración y desarrollo que permite a los desarrolladores interactuar y probar el funcionamiento de servicios web y aplicaciones, proporcionando una interfaz gráfica intuitiva y fácil de usar para enviar solicitudes a servidores web y recibir las respuestas correspondientes.

Con esta plataforma se pueden gestionar diferentes entornos de desarrollo, organizar las solicitudes en colecciones y realizar pruebas automatizadas para verificar el comportamiento de los sistemas.

Postman es utilizado por los desarrolladores para testear colecciones y catálogos APIs para gestionar el ciclo de vida de las estas, mejorar el trabajo colaborativo y mejorar la organización del proceso de diseño y desarrollo.

Para esta implementación en concreto, esta herramienta ayudó a escalar el proceso de conexión entre la aplicación y la web de Hugging Face. Una vez comprobado la correcta integración, se pasó a resolver las dificultades integradas al conectar la conexión web.



Figura 8- Postman logo

4.4.2 VS Code

[14] Este IDE es ampliamente conocido y utilizado por usuarios que utilizan la programación para diversas aplicaciones. Esta herramienta soporta gran variedad de lenguajes de programación distintos, librerías, frameworks, tiene inteligencia artificial integrada y demás funciones y utilidades.

Fue diseñado por Microsoft y gracias a su fácil uso y opciones como autocompletado y refactorización, lo hacen uno de los entornos más utilizados, por no decir el que más. Una de sus ventajas es su sistema de extensiones, para añadir complementos a tu trabajo y dar formato al código de manera uniforme, utilizar temas personalizados para mejorar la legibilidad y herramientas de depuración que facilitan y simplifican la detección y corrección de errores durante el desarrollo del código.

Para esta aplicación, VS Code sirvió como entorno utilizado para almacenar los diferentes scripts que la forman. A través de la extensión de Flutter, se dio lugar al uso de su lenguaje integrado (Dart) y diversas librerías que incluían funciones vitales para la ejecución. Este IDE contiene además una terminal incorporada, que permite ejecutar comando y scripts sin necesidad de salir del editor, agilizando así las tareas de desarrollo y gestión del proyecto.

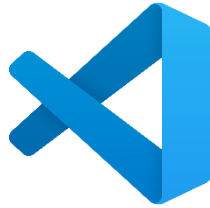


Figura 9- Visual Studio Code logo

4.4.3 Git/GitHub

Estas herramientas son útiles para llevar de manera controlada las actualizaciones del código a medida que se va avanzando en este e incluyendo funcionalidades y características más avanzadas, para tener un registro de versiones antiguas, facilitando su recuperación y control de errores.

Evita pérdidas de información, asegura un historial claro y facilita el desarrollo modular.

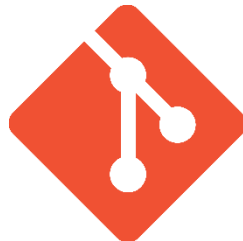


Figura 10- Git logo

4.5 Otros

4.5.1 JSON

JSON es un formato basado en texto para almacenar e intercambiar datos de una manera que es legible por humanos y analizable por máquina. Es normalmente usado por los desarrolladores para transferir datos entre un servidor y una aplicación web.

La naturaleza independiente del lenguaje lo convierte en ideal para intercambiar datos a través de diferentes lenguajes de programación y plataformas.

En el contexto del desarrollo, los tipos de dato son los diferentes tipos de valores que se pueden almacenar y manipular en un lenguaje de programación. Cada tipo de dato tiene su propio juego de atributos y comportamientos.

4.5.2 NPM

La principal función es facilitar la instalación, actualización y gestión de bibliotecas, mejorando el flujo de trabajo en proyectos de software. Tiene gran relevancia dentro del ecosistema de Node.js y JavaScript actuando como gestor de paquetes y administrador de dependencias.

Con NPM, se tiene acceso a una amplia fuente de paquetes de código abierto, y hace posible añadir módulos de terceros sin necesidad de programar funcionalidades desde cero. Para más información, hace posible definir y ejecutar scripts personalizados, como la ejecución de pruebas, compilación o despliegue del proyecto.

En este proyecto, NPM se ha utilizado principalmente para gestionar las dependencias de Cloud Functions, asegurando la correcta instalación de paquetes como esenciales para la comunicación entre la aplicación y los servicios de Hugging Face.

4.5.3 HTTP/HTTPS

El protocolo de transferencia de hipertexto es un protocolo o conjunto de reglas de comunicación para la comunicación cliente-servidor. Cuando visita un sitio web, su navegador envía una solicitud HTTP al servidor web, que responde con una respuesta HTTP. El servidor web y su navegador intercambian datos como texto sin formato. En resumen, el protocolo HTTP es la tecnología subyacente que impulsa la comunicación de red.

Sin embargo, este protocolo transmite datos no cifrados, lo que significa que la información enviada desde un navegador puede ser interceptada y leída por terceros. Este no era el proceso ideal, por lo que se extendió a HTTPS para agregar otro nivel de seguridad a la comunicación. HTTPS combina las solicitudes y respuestas HTTP con la tecnología SSL y TLS.

El manejo de solicitudes, respuestas y estados de conexión característicos de estos protocolos (GET, POST, 200) han sido primordiales durante el intercambio de datos entre la aplicación Flutter con Firebase Authentication y Firestore, además de las conexiones con Hugging Face.

4.5.4 Draw.io

Esta herramienta de diseño permite generar bocetos, diagramas y estructuras con flexibilidad y adaptabilidad. Con draw.io se ha conseguido generar una serie de plantillas e imágenes que serán utilizadas a lo largo de la memoria, permitiendo una mejor visualización de los conceptos y mejorando la claridad y el entendimiento de las explicaciones.

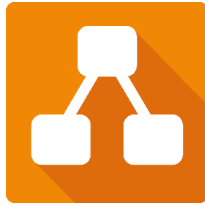


Figura 11- Logo Draw.io

4.5.5 PlantUML

PlantUML permite el desarrollo de gráficos secuenciales a partir de un código sencillo. Primeramente, se definen las identidades que participarán el intercambio de mensajes o información y posteriormente se procede a interconectar dichos elementos mediante flechas e instrucciones sencillas para generar diagramas. Esta herramienta ha sido útil para representar el flujo de datos que se traspasa entre los diferentes módulos que componen el proyecto y que también se exponen más adelante.

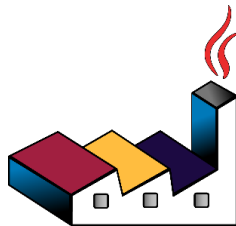


Figura 12- Logo PlantUML

4.5.6 Mermaid Live Editor

Editor web de creación de pizarras digitales para mostrar flujos entre diferentes entidades. En esta aplicación, se usará este editor más adelante para mostrar el flujo de navegación existente entre las diferentes ventanas que contiene dicha app. Utiliza pseudocódigo para generar instrucciones y elaborar diagramas a partir de las mismas. Es una herramienta útil para presentar un diseño que refleje unas ideas claras de forma rápida.



Figura 13- Mermaid Live Editor logo

4.5.7 Canva

Canva es un software creado para diseño, tiene una plataforma que proporciona herramientas para crear gráficos, presentaciones, postales, productos promocionales y sitios web para redes sociales. Sus numerosas plantillas, formatos de construcción, accesibilidad y componentes hace que sea una utilidad usada por muchos usuarios. Canva sirvió en este proyecto para elaborar un logo personalizado para la aplicación, lo que dio lugar a un toque más profesional, original y auténtico.



Figura 14- Canva logo

Capítulo 5. Análisis del Sistema

Este capítulo de la memoria pretende caracterizar el sistema, ofreciendo una descripción general del comportamiento de este, junto con una serie de utilidades que lo identifican. A continuación, se presentan los apartados mencionados junto con los requisitos funcionales y no funcionales del sistema.

5.1 Descripción general

El sistema propuesto se corresponde con una aplicación de gestión 'CRUD' combinada con los dotes de Inteligencia Artificial para ofrecer un avance y mejora en la gestión de pacientes, médicos y los diagnósticos que se ven involucrados en el proceso de consultas.

Está diseñada para tener un control total de la gestión de una base de datos actualizable, transcribir información oral a texto y generar diagnósticos y recomendaciones a dicho texto específico con ayuda de la IA para obtener un proceso controlado, fiable, rápido y seguro. La arquitectura del sistema se compone de los siguientes tres módulos principales:

❖ **Módulo Frontend:**

Esta sección se corresponde con el código procesado en Flutter y representa la interfaz con la que interactúa el usuario. Las funciones principales que identifican este módulo son las siguientes:

- Permitir autenticación de usuarios mediante verificación de credenciales.
- Navegar entre webs externas y páginas de navegación de la propia aplicación.
- Añadir, actualizar y eliminar tanto pacientes como usuarios que necesiten estar almacenados en la base de datos
- Gestionar la carga y visualización de documentos médicos en formato texto.
- Convertir voz a texto en tiempo real para consultas médicas.
- Enviar solicitudes a la IA y mostrar los diagnósticos generados.

❖ **Módulo Backend:**

Como se ha explicado en capítulos previos, este sistema no contiene como tal un apartado backend clásico como otras aplicaciones tradicionales, sino que posee una arquitectura sin un servidor localizado, lo que hace más escalable y fácil de manejar usando herramientas como Functions de Firebase y Node.js. Las responsabilidades más notables de este módulo son:

- Procesamiento de solicitudes HTTP provenientes y entrantes a la aplicación.
- Envío de información médica a los modelos alojados en Hugging Face.
- Recepción y formateo de las respuestas de los modelos de IA antes de devolverlas al Frontend.
- Gestión de lógica sin mantenimiento de servidores.

❖ **Base de Datos:**

Para el almacenamiento y gestión de usuarios se ha utilizado la función de Cloud Firestore, una base de datos no relacional (NoSQL) proporcionada por Firebase.

El hecho de que no haya relación entre colecciones hace más fácil su uso. Al no tener dependencias entre documentos, atributos o elementos, puedes editar, añadir o eliminar sin estar pendiente de que la acción pueda acarrear consecuencias en otras partes de la base de datos.

Por otra parte, también se está usando almacenamiento en Firebase Storage, cuya función es asegurar los documentos que se suben para ser almacenados. Las funciones de estas bases de datos en el proyecto son:

- Guardar un registro del historial de pacientes y los respectivos médicos activos.
- Apilar consultas médicas y documentos analizados.
- Gestionar el historial de transiciones entre los usuarios y el sistema.
- Mantener la información accesible en tiempo real para mejorar la experiencia del usuario.

5.2 Requisitos funcionales

Los requisitos funcionales son declaraciones de los servicios que prestará el sistema, en la forma en que reaccionará a determinados insumos. Describen lo que el sistema debe hacer y definen las funcionalidades y características esenciales que debe cumplir la aplicación para que cumpla su propósito.

Requisitos Generales:

- RF1: Los usuarios pueden autenticarse mediante correo y contraseña.
- RF2: La aplicación detecta si hay error y notifica mediante un mensaje.
- RF3: Los usuarios pueden recuperar su contraseña en caso de pérdida.
- RF4: Se permite cerrar la sesión y volver a la pantalla de login.

- RF5: Los usuarios autenticados pueden navegar entre pestañas distintas de la interfaz.
- RF5: Los usuarios autenticados pueden acceder a enlaces web externos.

Requisitos de Gestión de Pacientes y Médicos:

- RF7: Los usuarios autenticados pueden agregar nuevos pacientes y médicos.
- RF8: Los usuarios autenticados pueden editar pacientes y médicos existentes.
- RF9: Los usuarios autenticados pueden eliminar pacientes y médicos existentes.

Requisitos de Documentación:

- RF10: Los usuarios pueden cargar texto médico en la plataforma.
- RF11: La aplicación puede analizar el texto cargado y devolver una respuesta generada.
- RF12: El sistema permite visualizar las respuestas a través de la interfaz.
- RF13: El sistema notifica con mensajes de éxito la subida de archivos.

Requisitos de Conversión de Voz a Texto:

- RF14: La aplicación permite seleccionar un paciente existente para personalizar el tratamiento
- RF15: La aplicación permite grabar voz en tiempo real.
- RF16: La aplicación puede transcribir la voz grabada a texto.
- RF17: La aplicación permite al usuario editar el texto convertido.
- RF18: Se permite generar una conversación/chat con la IA para consultas actuales y futuras.

Requisitos de interacción con IA:

- RF19: La plataforma puede enviar el texto de los documentos cargados al modelo alojado en Hugging Face.
- RF20: La plataforma puede enviar el texto transcrito final al modelo alojado en Hugging Face.
- RF21: La plataforma puede recibir las respuestas de la IA.
- RF22: La plataforma puede mostrar las respuestas al usuario en un formato comprensible.

5.3 Requisitos no funcionales

Los requisitos no funcionales son aquellos que no se refieren directamente a las funciones específicas suministradas por el sistema sino a las propiedades del sistema: rendimiento, seguridad, disponibilidad. No se enfocan en qué hace la aplicación (como los requisitos funcionales), sino en cómo debe hacerlo para garantizar una buena experiencia de usuario y un funcionamiento eficiente.

Usabilidad:

- RNF1: La aplicación debe ofrecer una interfaz intuitiva y fácil de usar.
- RNF2: La navegación debe ser clara, permitiendo al usuario acceder a las funciones principales sin dificultad.
- RNF3: La aplicación debe ofrecer retroalimentación visual cuando se realicen acciones importantes.

Seguridad y Privacidad:

- RNF4: Todos los pacientes y médicos deben almacenarse de forma segura en la base de datos.
- RNF5: La autenticación de usuarios debe realizarse mediante Firebase Authentication, garantizando que solo usuarios autorizados puedan acceder a los datos.
- RNF6: Los documentos de texto deben almacenarse de forma segura en Firebase Storage.
- RNF7: La comunicación entre el frontend y el backend debe realizarse mediante conexiones HTTPS cifradas, evitando accesos no autorizados.

Rendimiento y Eficiencia:

- RNF8: La conversión de voz a texto debe realizarse de forma rápida para ofrecer una experiencia fluida.
- RNF9: La carga y procesamiento de documentos debe realizarse de forma rápida para ofrecer una experiencia fluida.
- RNF10: Las solicitudes a páginas externas deben ser asíncronas para evitar bloqueos en la interfaz.

Fiabilidad:

- RNF11: En caso de error en el procesamiento de datos, la aplicación debe proporcionar mensajes de error claros y guiar al usuario.

Capítulo 6. Especificación del Sistema

6.1 Casos de Uso

Para la especificación del sistema resumiremos de forma conceptual las interacciones posibles entre los usuarios y el sistema. A continuación, se desarrollará dicha información en formato tabla, para que la organización sea más visual y las ideas queden claras y concisas:

ID	Título	Descripción	Actores	Flujo de Eventos
CU1	Autenticación de usuario	Para asegurar que el acceso a la aplicación está restringido únicamente a usuarios habilitados, se utiliza una pestaña de inicio de sesión a través de credenciales.	Usuario, Firebase Authentication, Aplicación.	1. El usuario introduce sus credenciales. 2. El sistema verifica la autenticidad. 3. Se concede acceso si las credenciales son correctas.

ID	Título	Descripción	Actores	Flujo de Eventos
CU2	Recuperación de contraseña	Para facilitar que la pérdida u olvido de contraseña no suponga un impedimento para el acceso, siempre que se verifique la identidad del usuario.	Usuario, Firebase Authentication, Email, Aplicación.	1. Se pulsa '¿Olvidaste tu contraseña?'. 2. Se introduce dirección de correo. 3. Se verifica el correo introducido. 4. Se reestablece la contraseña-

ID	Título	Descripción	Actores	Flujo de Eventos
CU3	Cierre de sesión	El usuario vigente puede cerrar la sesión activa sin necesidad de apagar la aplicación para dejar libre su uso a otro usuario.	Usuario, Aplicación.	1. Se hace clic en el icono de salida de la 'home page'. 2. Se confirma la solicitud de cierre. 3. La aplicación redirige al usuario convenientemente al 'login'.

ID	Título	Descripción	Actores	Flujo de Eventos
CU4	Navegación en la aplicación	De entre las distintas pestañas y utilidades que ofrece el sistema, el usuario es capaz de viajar entre ellas a su conveniencia, desplazándose para buscar la función que necesite.	Usuario, Aplicación.	1. Se mantiene el cursor sobre el elemento a seleccionar. 2. Se hace clic en la interfaz interactiva. 3. La aplicación redirige al usuario convenientemente.

ID	Título	Descripción	Actores	Flujo de Eventos
CU5	Generación de un nuevo paciente o médico.	El usuario podrá añadir tantas personas como necesite y serán almacenados y visibles en tiempo real.	Usuario, Firestore Database, Aplicación.	1. Se navega hasta la pestaña correspondiente en la aplicación. 2. Se hace clic en el botón de añadir. 3. Se teclean los atributos del nuevo integrante. 4. Se salvan los cambios.

ID	Título	Descripción	Actores	Flujo de Eventos
CU6	Edición de un paciente o médico.	El usuario podrá editar cualquier atributo de los personajes que haya añadido con anterioridad o ya existentes en la base de datos.	Usuario, Firestore Database, Aplicación.	1. Se navega hasta la pestaña correspondiente en la aplicación. 2. Se hace clic en el individuo a modificar. 3. Se cambian los atributos erróneos. 4. Se salvan los cambios.

ID	Título	Descripción	Actores	Flujo de Eventos
CU7	Eliminación de un paciente o médico.	El usuario podrá eliminar cualquiera de los personajes que haya añadido con anterioridad o ya existentes en la base de datos.	Usuario, Firestore Database, Aplicación.	1. Se navega hasta la pestaña correspondiente en la aplicación. 2. Se hace clic en el icono de papelera al lado de la persona a eliminar.

ID	Título	Descripción	Actores	Flujo de Eventos
CU8	Subir un documento de texto médico.	El usuario podrá cargar un documento de texto médico para que este quede almacenada o en Firebase Storage y se procese su análisis.	Usuario, Firebase Storage, Aplicación.	1. Se navega hasta la pestaña correspondiente en la aplicación. 2. Se hace clic en el selector de archivos. 3. Se elige de entre los archivos posibles del explorador.

ID	Título	Descripción	Actores	Flujo de Eventos
CU9	Procesamiento de documento	El sistema analiza el contenido del documento médico y extrae información relevante, para su posterior análisis.	Usuario, IA (Hugging Face), Firebase Functions	1. El usuario sube un documento. 2. Firebase Functions procesa el archivo y lo envía a Hugging Face. 3. Se genera un diagnóstico y se devuelve a la app.

ID	Título	Descripción	Actores	Flujo de Eventos
CU10	Seleccionar un paciente para adaptar los servicios.	El usuario podrá elegir de entre los pacientes existentes en el sistema para personalizar los servicios y adaptar los diagnósticos.	Usuario, Firestore Database, Aplicación.	1. Se navega hasta la pestaña correspondiente en la aplicación. 2. Se hace clic en el selector de pacientes. 3. Se confirma el paciente

ID	Título	Descripción	Actores	Flujo de Eventos
CU11	Grabación de voz	El usuario graba un audio para su posterior transcripción.	Usuario, Firebase Functions	1. Se navega hasta la pestaña correspondiente en la aplicación. 2. Se selecciona el paciente. 3. Se inicia la grabación pulsando el botón.

ID	Título	Descripción	Actores	Flujo de Eventos
CU12	Conversión de voz a texto	El sistema convierte el audio grabado en texto.	Usuario, Aplicación	1. Se detiene la grabación pulsando el mismo botón de grabar. 2. La aplicación convierte el audio en texto. 4. Se modifica el recuadro de texto generado si hay errores.

ID	Título	Descripción	Actores	Flujo de Eventos
CU13	Generación de diagnóstico con IA.	El sistema envía un texto procesado a Hugging Face para recibir un diagnóstico.	Usuario, IA (Hugging Face), Firebase Functions	1. Se envía el texto extraído. 2. Hugging Face analiza el contenido. 3. Se genera una respuesta con el diagnóstico.

ID	Título	Descripción	Actores	Flujo de Eventos
CU14	Visualización del diagnóstico.	El usuario puede ver el resultado del análisis en la interfaz o en Firestore.	Usuario, Firestore Database	1. El usuario accede a su historial. 2. Se cargan los diagnósticos almacenados con toda la información relevante.

6.2 Flujo de Información entre los módulos

El sistema está compuesto por varios módulos que se comunican entre sí para cumplir con las funcionalidades requeridas. De entre las partes que componen esta aplicación, es necesaria una coordinación y monitorización de ambas para que las operaciones exigidas tengan éxito.

Tras los distintos casos de usos expuestos con anterioridad, se hará un resumen ilustrativo que mostrará cómo se comporta internamente la aplicación al tener un constante flujo de información. Estas son las principales acciones llevadas a cabo entre módulos:

Autenticación y Gestión de Usuarios:

- El usuario administrador intenta acceder a la aplicación a través de un sistema de inicio de sesión.
- Firebase Authentication verifica las credenciales y devuelve un token / booleano dependiendo del éxito de la operación.
- Si la autenticación es exitosa, se permite el acceso al resto de funcionalidades.

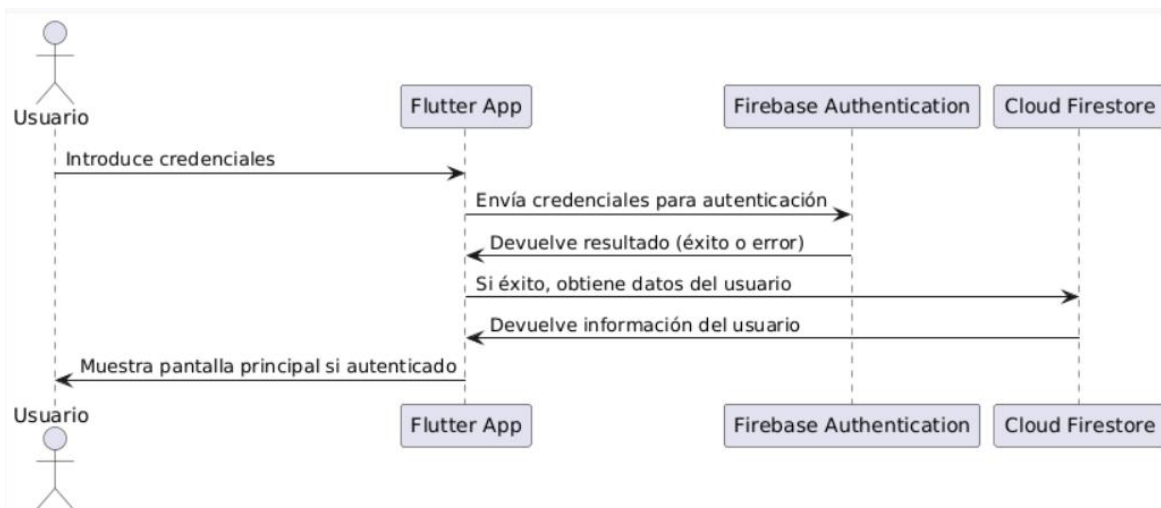


Figura 15- Diagrama de secuencia para la autenticación de usuario

Procesamiento de Documentos:

- El usuario carga un documento en la aplicación.
- El archivo se almacena en Firebase Storage.
- Firebase Functions procesa el archivo y lo envía a Hugging Face para su análisis.
- Hugging Face responde con un diagnóstico basado en IA.
- La respuesta se almacena en Cloud Firestore y se muestra al usuario.

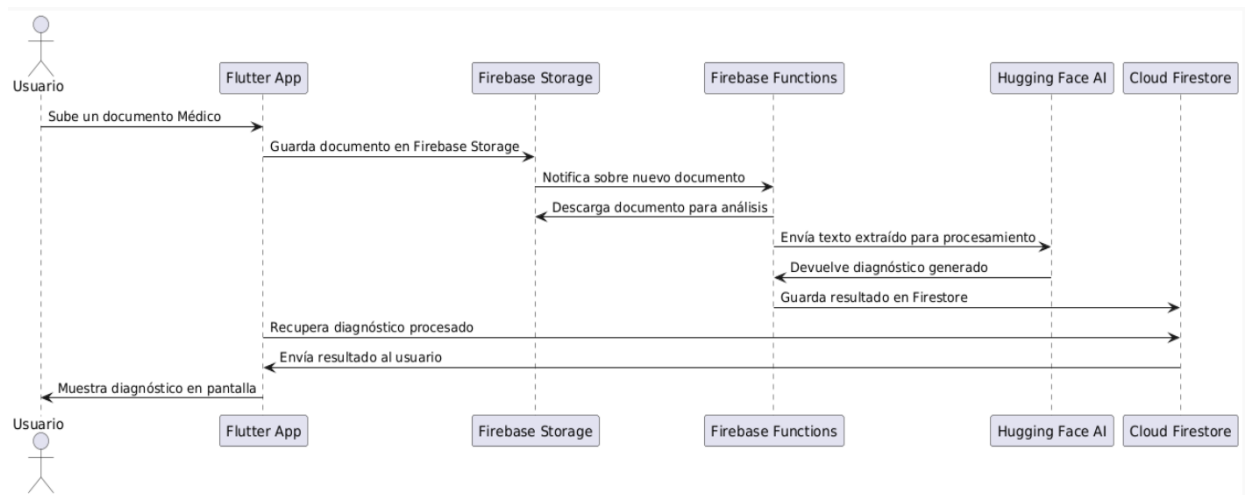


Figura 16- Diagrama de secuencia para el procesamiento de documentos

Conversión de Voz a Texto:

- El usuario inicia la grabación de voz en la aplicación.
- Flutter utiliza una librería de reconocimiento de voz para transcribir el audio en tiempo real.
- La transcripción se muestra en la interfaz de usuario, permitiendo a este editar el texto manualmente si es necesario.
- El usuario confirma la transcripción y la envía al modelo de IA en Hugging Face.
- La IA procesa el texto y devuelve un diagnóstico, que se almacena en Firestore.

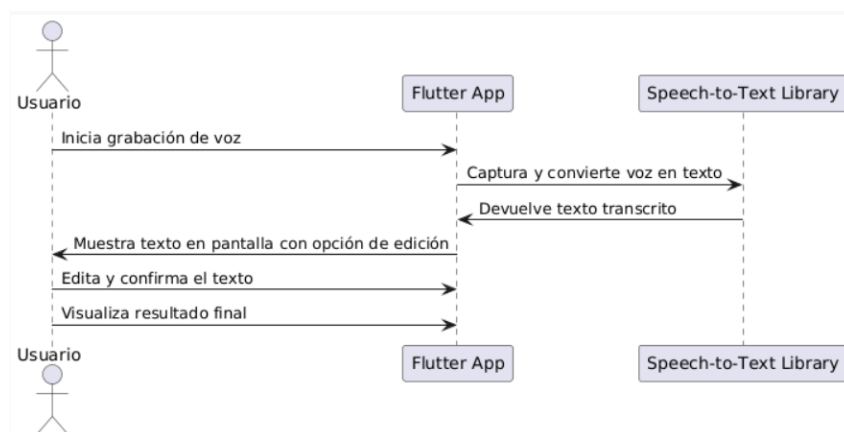


Figura 17- Diagrama de secuencia de conversión de voz

Generación de diagnósticos y visualización de resultados:

- El usuario elige entre un documento procesado o una transcripción de voz.
- La aplicación envía el texto a Hugging Face para su análisis.
- Hugging Face genera un posible diagnóstico y lo devuelve al usuario.
- El resultado se almacena en Firestore para su consulta posterior.

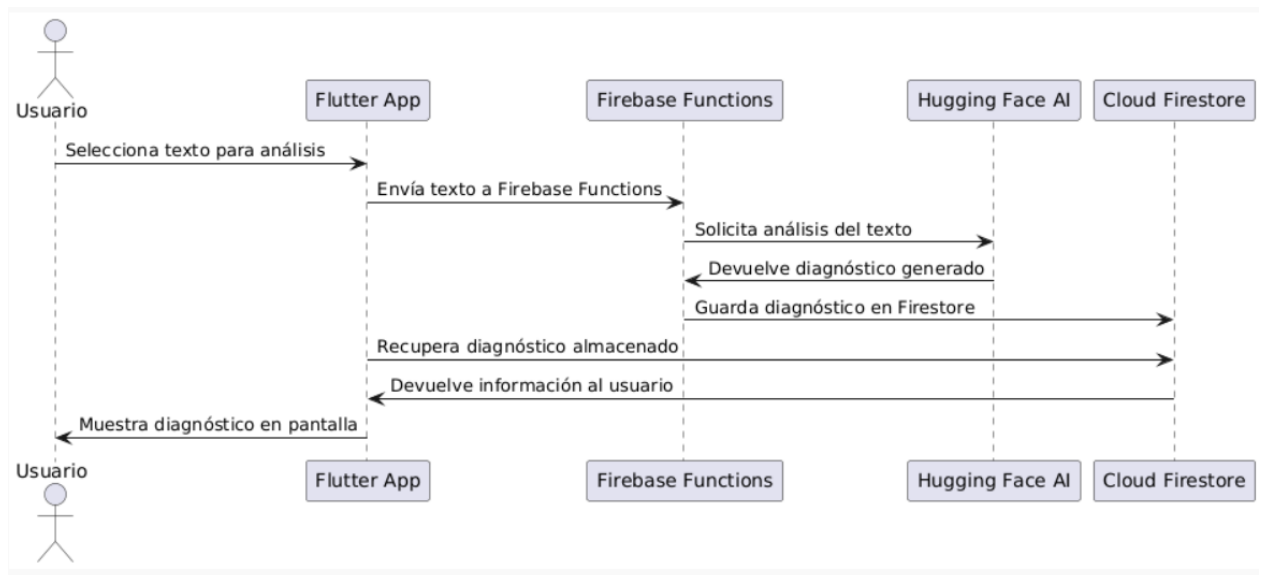


Figura 18- Diagrama de secuencia para visualización de diagnósticos

Capítulo 7. Diseño del Sistema

7.1 Arquitectura General

La arquitectura general del sistema se representa de la siguiente forma:

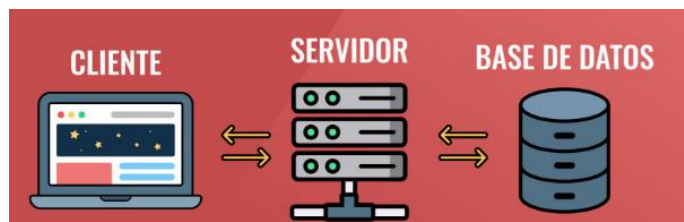


Figura 19- Esquema general del diseño del sistema

En la figura 19 se puede apreciar las partes que componen la aplicación y que ya se han descrito previamente, además la parte central del servidor es una simulación gráfica sobre el comportamiento general del sistema, aunque ya se ha aclarado el funcionamiento 'servless'.

En la arquitectura de la aplicación, se ha implementado un enfoque modular y flexible a través del uso de microservicios. En lugar de alojar todas las partes de la aplicación en un solo servidor, se han dividido en componentes independientes, cada uno representado por un microservicio:

1. Frontend (Flutter)

Es la interfaz del usuario (cliente) de la aplicación, desarrollada en Flutter para garantizar compatibilidad multiplataforma. Permite que los usuarios:

- ✓ Accedan al sistema mediante Firebase Authentication.
- ✓ Suban y visualicen documentos de texto médico.
- ✓ Graben voz y obtengan una transcripción en tiempo real.
- ✓ Interactúen con los diagnósticos generados por la IA.
- ✓ Consulten registro sobre el historial de pacientes y médicos
- ✓ Naveguen entre diferentes webs disponibles.

2. Backend (Firebase Functions + Node.js)

El backend es 'serverless', su ejecución interviene asincrónicamente cuando se le envía una solicitud, permitiendo escalabilidad y una adecuada gestión de recursos. Se encarga de:

- ✓ Procesar solicitudes enviadas desde el frontend.
- ✓ Interactuar con Hugging Face para enviar texto y recibir diagnósticos médicos.
- ✓ Gestionar la seguridad en el acceso a los datos.

3. Base de Datos (Cloud Firestore y Firebase Storage)

Este módulo es responsable del almacenamiento y gestión de datos médicos. Se compone de:

- ✓ Cloud Firestore: Almacena información estructurada como usuarios, pacientes y consultas médicas.
- ✓ Firebase Storage: Guarda documentos médicos.

Además, cabe destacar que:

- La comunicación entre bloques ha de ser segura.
- El intercambio de datos se realiza haciendo uso de JSON.
- El navegador cliente debe de poder ejecutar JavaScript.

7.2 Diseño de la Base de Datos

Como se ha mencionado anteriormente, la base de datos en este caso se comporta como un sistema de almacenamiento NoSQL que organiza la información en colecciones, documentos y sus respectivos atributos. Esto lo convierte en una solución eficiente, permitiendo una sincronización en tiempo real con la app en Flutter.

La elección de Cloud Firestore como base de datos en este proyecto tiene que ver con las siguientes características:

- ✓ Escalabilidad: Soporta múltiples usuarios simultáneos sin perder rendimiento.
- ✓ Acceso en tiempo real: La información se actualiza automáticamente tras ser modificada en la aplicación.
- ✓ Seguridad
- ✓ Flexibilidad: No requiere una organización estricta.



Figura 20- Diagrama entidad-relación

Como podemos apreciar en la Figura anterior, los datos se organizan en colecciones, cada una conteniendo documentos con distintos atributos.

✧ Colecciones y documentos:

Pacientes

- Nombre
- DNI
- Fecha de nacimiento
- Teléfono
- Localidad
 - MENSAJES
 - Contenido
 - Emisor
 - Fecha

Médicos

- Nombre
- Especialidad
- Fecha de nacimiento
- Teléfono
- Localidad

Transcripciones

- Diagnóstico
- Nombre del paciente
- Problema
- Fecha
- Usuario administrador

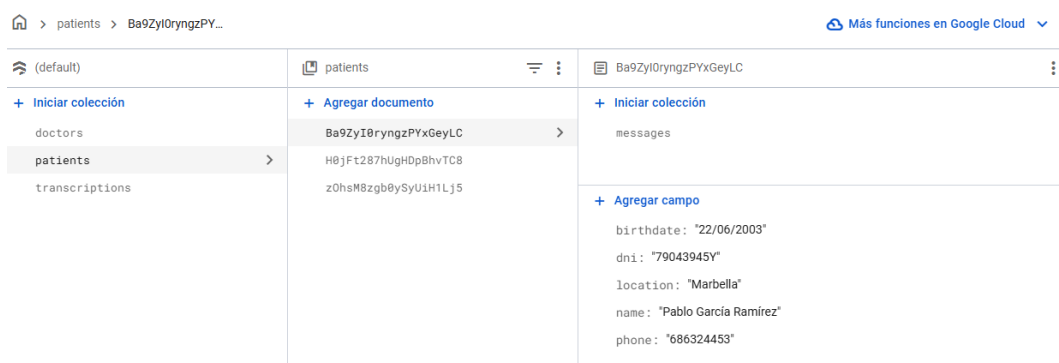


Figura 21- Ejemplo de colecciones reales en Firestore database

7.3 Estructura del módulo Backend

Acerca del diseño de este módulo del proyecto, se explicará de qué forma está conectado este esquema modular según distintas funcionalidades que utiliza la aplicación y acompañada de imágenes que ilustren la composición y el proceso.

Esta parte está formada por Firebase Functions en combinación con Node.js, permitiendo realizar procesamiento de datos, integraciones con IA y gestión de la base de datos sin necesidad de administrar infraestructura propia. El módulo en sí se compone de los siguientes elementos clave:

- ✓ Firebase Functions: Ejecuta la lógica y se comunica con la base de datos.
- ✓ Node.js: Entorno de JavaScript que manejará las peticiones web.
- ✓ Axios: Cliente HTTP encargado de enviar solicitudes.
- ✓ Firebase Admin SDK: Biblioteca que habilita la interacción.

Función	Activador	Versión	Solicitudes (24 h)	Cantidad mínima/máxima de instancias	Tiempo de espera
<code>analyzeMedicalReport</code> us-central1	HTTP Solicitud <code>https://analyzemedicalreport-kywquofja-uc.a.run.app</code>	v2	0	0 / 3	1 min

Figura 22- Ejemplo real de Firebase Functions

Con anterioridad se expusieron ejemplos prácticos de acciones reales que ejecutaba la aplicación para observar el flujo de información entre módulos, de manera secuencial, a continuación, se mostrarán los pasos que seguiría este módulo concreto en tiempo de ejecución:

1. Flutter envía un texto transcrito al backend mediante Firebase Functions.
2. Firebase Functions recibe la solicitud y procesa el texto.
3. Firebase Functions envía el texto a Hugging Face para su análisis.
4. Hugging Face devuelve un diagnóstico basado en IA.
5. Firebase Functions almacena el resultado en Cloud Firestore.
6. Flutter recupera la información de Firestore y la muestra al usuario.

En el caso de subir informes médicos con texto a ser analizado, el proceso es muy similar, sólo que dicho documento también queda almacenado en un apartado de Firebase llamado Storage, las siguientes imágenes lo esbozan con claridad:



Figura 23- Carpeta de almacenamiento de Firebase Storage

gs://flutter-crud-a718e.firebaseiostorage.app > reports					Subir archivo		
☐	Nombre	Tamaño	Tipo	Modificación más reciente			
☐	 1742982224062_Sample_report.txt	57 B	application/octet-stream	26 mar 2025			

Figura 24- Documento almacenado en la carpeta tras ser analizado

7.4 Estructura del módulo Frontend

El principal objetivo del frontend es proporcionar una experiencia de usuario fluida, esto se consigue a través de una serie de operaciones exitosas como: la gestión de usuarios que componen el sistema, el control de documentos almacenados en la nube, las transcripciones realizadas, el respectivo análisis de la inteligencia artificial, la visualización de los resultados...

7.4.1 Arquitectura interna del Frontend

El programa está compuesto por diferentes scripts de código que ensamblados cumplen con unas funcionalidades globales. A la hora de realizar este tipo de aplicaciones, lo óptimo es reorganizar la arquitectura para tener un sistema distribuido, limpio y eficiente.

Para conseguir esto, podemos identificar una serie de capas, cumpliendo cada una con una serie de tareas específicas. Por ejemplo, podemos diferenciar las siguientes:

- ➔ **Capa de Presentación:** Esta será la encargada de gestionar y monitorizar todo lo referente a la estética de la aplicación, es decir, se encarga de mantener una interfaz amigable y vistosa para que el usuario pueda manejarla con facilidad.

Los widgets que la componen están orientados a usar colores, formas, estilos de texto, rellenos, iconos y demás componentes para lograr este tipo de resultados.

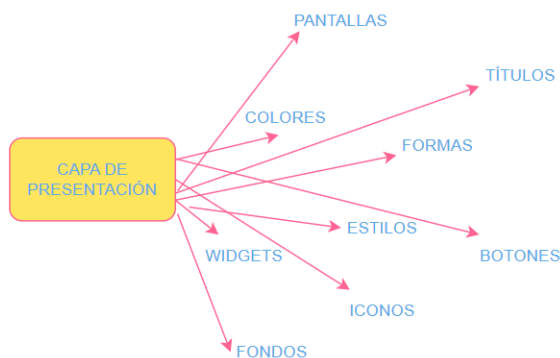


Figura 25- Capa de Presentación

- ➔ Capa de Gestión de Estado: La capa de gestión de estado manipula el flujo de información entre los componentes. Según mi implementación, la información que se muestra en la app es cambiante, por lo que debe actualizarse a medida que se van introduciendo cambios.

Esta capa contiene dos principios fundamentales: la validación y la verificación. Estos conceptos refieren a que la aplicación funcione como está pensado que lo haga y, además, libre de errores.



Figura 26- Capa de Gestión de Estado

- ➔ Capa de Servicios: Debido a la variedad de componentes que engloban esta aplicación, es necesaria una capa de servicios. Esta, a diferencia de la anterior, está pensada para dedicarse a servicios concretos, como por ejemplo la autenticación.

Cada parte del código de esta capa está pensada para un servicio diferente, de forma que funcione de forma aislada y pueda utilizarse en cualquier parte del proyecto una vez requerido.

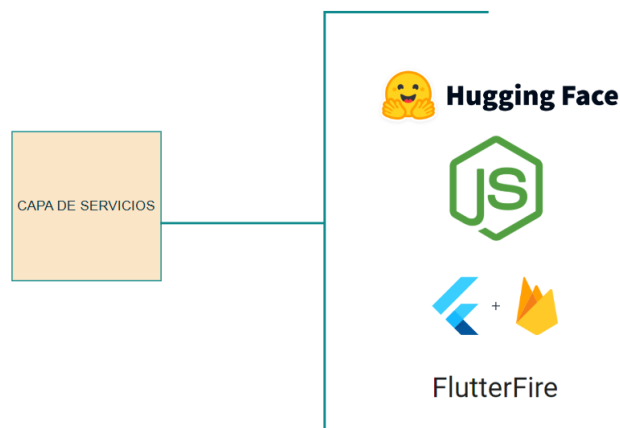


Figura 27- Capa de Servicio

7.4.2 Estructura de Carpetas y Organización del Código

Es bien sabido que, al realizar proyectos grandes con varias características, una buena práctica para los programadores viene dada por una división del código. Al tener un programa grande inicial, lo más sensato es dividirlo modularmente para separar los problemas y escalarlos a un nivel más sencillo.

Se ha seguido una fragmentación adecuada para estandarizar los diferentes elementos que componen la app, tener una estructura más nítida y sólida, y favorecer la depuración y corrección de errores.

A continuación, se expondrá de forma visual la estructura del código contenido en esta app en las siguientes 5 figuras, resaltando las diferentes carpetas y scripts por los que está compuesta:

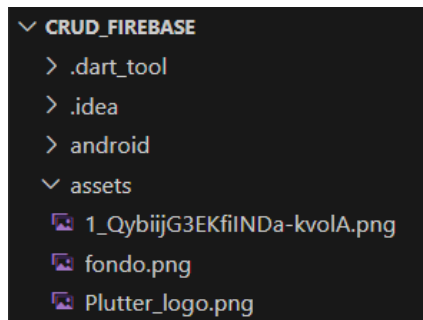


Figura 28- Carpeta 'assets' Flutter project

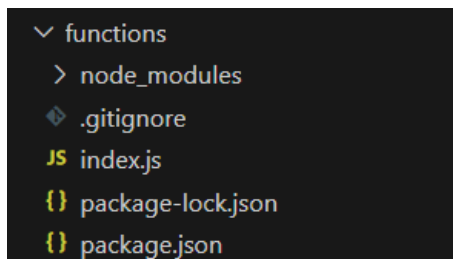


Figura 29- Carpeta 'functions' Flutter project

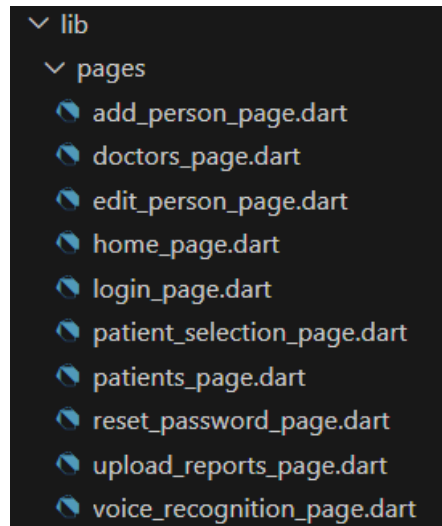


Figura 30- Carpeta 'pages' Flutter project

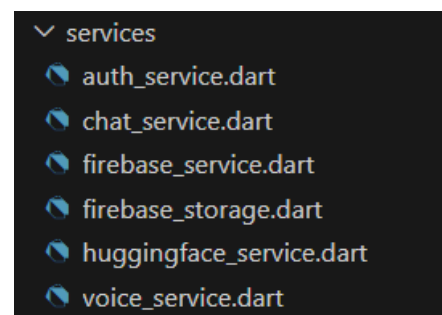


Figura 31- Carpeta 'services' Flutter project

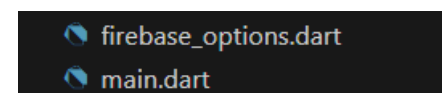


Figura 32- Scripts restantes Flutter project

Las ilustraciones anteriores representan un árbol generado en VS Code, nuestro IDE. La carpeta global 'lib' engloba las subcarpetas y los scripts contenidos en ellas. Como se puede observar, los scripts están escritos en el código de programación usado por Flutter, ya que poseen dicha extensión al final de cada archivo.

Para las funcionalidades indicadas, se ha decidido por una división entre servicios, páginas y una extra de funciones o 'functions'. Para cada pestaña disponible que pueda abrirse en la aplicación, hay un script específico para ella donde queda registrado su diseño y sus funcionalidades. Estas partes del código tienen la terminación '_page' en el nombre, haciendo referencia a que se trata de una pestaña de navegación. Esto es útil porque aísla las diferencias entre cada script, pudiendo personalizar los detalles de cada una.

Hay un total de 10 y es posible seleccionar cada una de ellas a través de la aplicación cuando esta está ejecutándose. No todas están disponibles desde la barra principal, sino que es posible que algunas queden derivadas cuando se activa cierta funcionalidad dentro de una pestaña concreta. Un ejemplo práctico sería el siguiente:

Si por ejemplo me ubico en la pantalla de pacientes activos, en la que se muestra una lista de los pacientes registrados disponibles en tiempo real y, quiero editar un error en el nombre o documento nacional de identidad de alguno de ellos, tan sólo tengo que pulsar sobre uno para que se abra la nueva pestaña de edición. Tras guardar el resultado, se vuelve a redireccionar a la pantalla inicial de pacientes activos.

Por otro lado, existe la carpeta de servicios. Esta es más técnica y compleja que la anterior, ya que trata de las funciones internas que realiza la aplicación. Similarmente, se decidió por una división de los servicios para atajar los problemas de forma modular. Hay un servicio por cada operación disponible, desde la autenticación inicial al abrir la app, gestión CRUD de personas en tiempo real con actualización directa en Firebase, control de las interacciones con la inteligencia artificial...

En esta carpeta hay un total de 6 clases, cada cual está pensado para actuar de una forma distinta y en los que hay una serie de funciones que son usadas en otras partes del código, como en los scripts de páginas. Básicamente, este tipo de documentos utilizan librerías para conectarse con módulos externos, comentados anteriormente, para que exista una comunicación fluida. Estas partes de código son reutilizables.

Además, en la Figura 29 se puede ver la estructura de la carpeta 'functions'. Esta carpeta contiene el script necesario, 'index.js', para definir las funciones usadas en Firebase Functions. También se trata de un script técnico, sin diseño alguno, escrito en JavaScript ya que es el encargado de procesar las peticiones generadas por la aplicación Flutter y conectarse con Hugging Face.

En esta misma carpeta podemos encontrar las dependencias y configuraciones del proyecto de Node.js. Es importante porque aquí se registran las librerías que utilizas, como 'firebase-functions' y 'node-fetch', necesarias para la construcción del código. También hay otros archivos generados automáticamente cuando se instalan paquetes, pero no han sido necesario modificarlos ni, por lo tanto, tampoco comentarlos.

La primera imagen muestra una carpeta con tan sólo 3 archivos, esta contiene imágenes que han sido integradas en la aplicación, pero que deben estar presentes en el proyecto para que la compilación se ejecute sin errores.

Finalmente, pasamos a hablar de los 2 últimos scripts, pero no menos importantes. Por una parte, tenemos el archivo principal o 'main script', este es el que se ejecuta al lanzar el proyecto y si no está bien configurado la aplicación no corre. Este documento es el punto de entrada a la aplicación, inicializa Firebase, configura estados globales y define la estructura de navegación. Junto a él se encuentra un documento con opciones que se configura al añadir Firebase al proyecto.

Sin dejarnos nada atrás, este último archivo del proyecto es esencial, ya que cada librería y dependencia que se ha ido añadiendo a medida que la aplicación se iba formando, tiene que estar presente en el archivo. Asimismo, las imágenes que comentábamos anteriormente aparecen en este documento y se procura que las versiones estén actualizadas para no dar lugar a problemas externos.

7.4.3 Navegación

La aplicación permite a los usuarios moverse entre distintas pantallas de forma intuitiva. Se ha implementado un sistema de rutas con 'Navigator.pushNamed()', asegurando una navegación fluida entre los módulos. Estas son las principales rutas de la aplicación:

1. **Pantalla de Inicio de Sesión**
2. **Pantalla de Recuperación de contraseña**
3. **Pantalla Principal (/home)**
4. **Pantalla de Pacientes (/patients)**
5. **Pantalla de Doctores (/doctors)**
6. **Pantalla de Subida de Archivos (/upload_reports)**
7. **Pantalla de Reconocimiento Auditivo (/voice_recognition)**

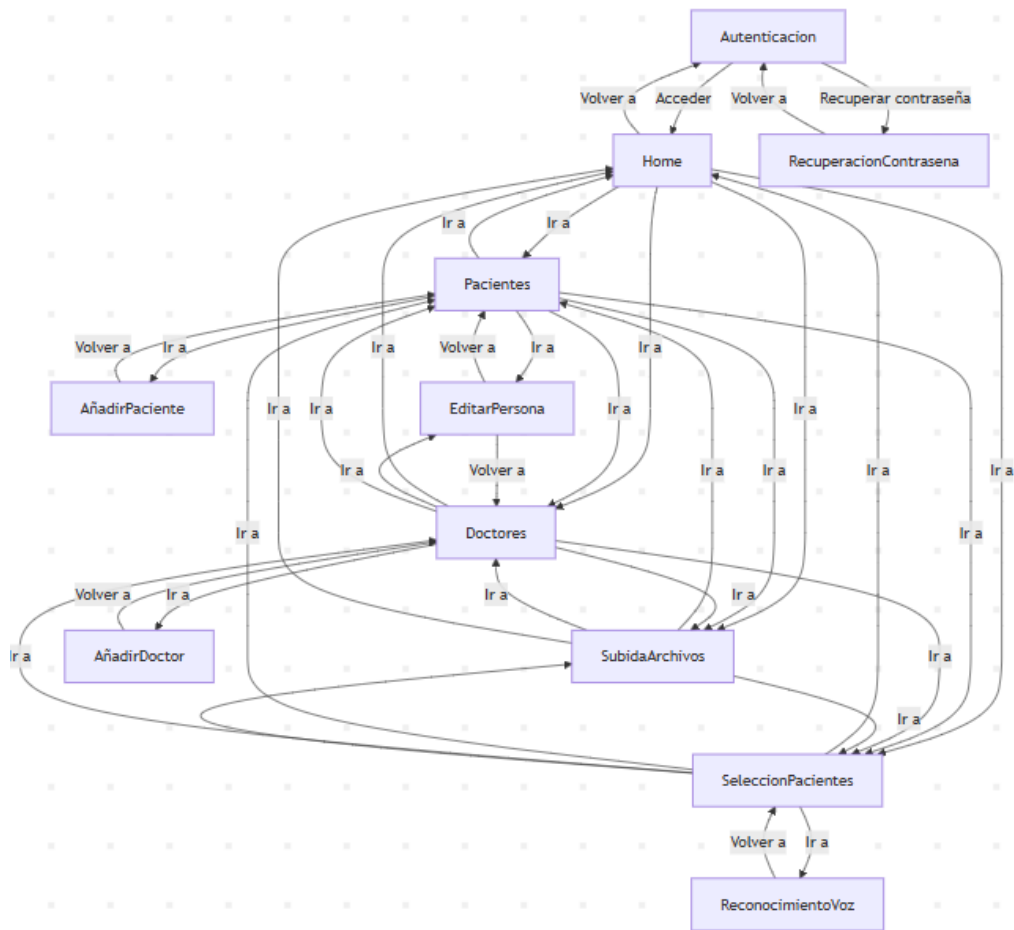


Figura 33- Diagrama de navegación entre ventanas

En la imagen anterior se muestra un diagrama completo de todas las posibles rutas y combinaciones que se pueden generar estando dentro de la aplicación. Ahí aparecen más ventanas de las principales nombradas anteriormente y es posible visualizar el flujo de navegación entre todas ellas.

En el siguiente capítulo se comentará más exhaustivamente la estructura y uso del código y cómo este interviene en las funciones que ejecuta la aplicación.

Capítulo 8. Implementación

8.1 Gestión y desarrollo interno

Durante la evolución de este capítulo y sus respectivos apartados se irán comentando los aspectos relacionados con la construcción del código. Se detallarán las funcionalidades conseguidas en la aplicación a través de las instrucciones programadas, y dicha información será apoyada con ilustraciones y comentarios que realzarán de una forma más visual el contenido y facilitarán un mejor entendimiento de este módulo más interno.

8.1.1 Módulo de gestión de usuarios

En esta memoria se ha dado un enfoque a cómo funciona este módulo o de qué está encargado. El acceso seguro a la plataforma con sus respectivos servicios y funcionalidades está soportado por un sistema de autenticación de usuario. Como es bien sabido, esto sólo autoriza a los administradores con un email y contraseña válidos y previamente registrados, a utilizar los numerosos servicios que presta la aplicación.

Cuando un usuario intenta iniciar sesión en la aplicación, sus credenciales son enviadas a Firebase Authentication y dependiendo del éxito o no de la operación, se permite el acceso al usuario o se devuelve un error (transmitido como un mensaje en la pantalla de inicio de sesión). Esto está desarrollado en la carpeta de servicios que fue detallada anteriormente. El código es el siguiente:

```
import 'package:firebase_auth/firebase_auth.dart';

FirebaseAuth auth = FirebaseAuth.instance;

Future<User?> signIn(String email, String password) async{
  try{
    UserCredential credenciales =
      await auth.signInWithEmailAndPassword(email: email, password: password);
    return credenciales.user;
  }catch(e){
    print("Error al iniciar sesión: $e");
    return null;
  }
}
```

Figura 34- Código: Función para inicio de sesión

La importación de las librerías es reiterativa a lo largo de los scripts que abordan la aplicación y su operabilidad es esencial.

A consecuencia de esto, se permite el uso de ciertas 'clases', también implementadas con interioridad, cuyas instancias hacen que podamos usar las funcionalidades que dichas clases poseen.

Una funcionalidad añadida del inicio de sesión es la pestaña de recuperación de contraseña, que juega un papel importante en cualquier aplicación o página web que necesita de credenciales. Debido a la variedad de plataformas que exigen un inicio de sesión con credenciales privadas, el riesgo de pérdida u olvido de estos datos aumenta.

Como solución, se presenta un apartado en el que el usuario puede recuperar de una forma fácil, rápida y segura sus datos de acceso para reestablecer los códigos que lo identifican. Gracias a Firebase authentication, el sistema envía un email de verificación al usuario con un enlace para resolver este inconveniente.

```
import 'package:flutter/material.dart';
import '../services/auth_service.dart';

class ResetPasswordPage extends StatefulWidget {
  const ResetPasswordPage({super.key});

  @override
  ResetPasswordPageState createState() => _ResetPasswordPageState();
}

class _ResetPasswordPageState extends State<ResetPasswordPage> {
  final TextEditingController _emailController = TextEditingController();
  String _message = '';

  Future<void> _resetPassword() async {
    try {
      await resetPassword(_emailController.text.trim());
      setState(() {
        _message =
          "📧 Se ha enviado un correo para reestablecer tu contraseña.";
      });
    } catch (e) {
      setState(() {
        _message = "⚠️ Error al enviar el correo. Verifica tu dirección.";
      });
    }
  }
}
```

Figura 35- Código: Función para reestablecer contraseña

Se puede observar de manera rápida un esquema que comparten numerosos scripts de esta aplicación, en el que primeramente se definen los parámetros de pestaña y constructores, y posteriormente la definición de la función controladora del servicio. Se han utilizado mensajes y emojis para facilitar una mejor experiencia de usuario, el resto del código no aparece en la imagen, pero está dedicado a diseño gráfico.

Además de la autenticación, los datos de los usuarios deben almacenarse y gestionarse adecuadamente en Firestore. Esto permite funcionalidades como la recuperación de información de cada usuario y la asignación de roles.

Este modelo de aplicación se denomina CRUD, un acrónimo proveniente del inglés cuyas siglas corresponden a: Create, Reade, Update y Delete. La respectiva traducción quedará registrada en la sección de acrónimos de la memoria.

A continuación, se mostrarán una serie de capturas de pantalla provenientes del código almacenado en nuestro IDE que están dedicadas a resolver los requerimientos de este tipo de gestión CRUD de personas:

```
// Escuchar cambios en la colección de pacientes en tiempo real
Stream<List<Map<String, dynamic>>> getPatientsStream() {
    return firestore.collection('patients').snapshots().map(
        (snapshot) => snapshot.docs.map((doc) {
            return {
                "uid": doc.id,
                "name": doc["name"],
                "dni": doc["dni"],
                "birthdate": doc['birthdate'],
                "phone": doc['phone'],
                "location": doc['location'],
            };
        }).toList(),
    );
}
```

Figura 36- Código: Función para leer pacientes en tiempo real

```
// Agregar paciente
Future<void> addPatient(String name, String dni, String birthdate, String phone,
    String location) async {
    await firestore.collection("patients").add({
        "name": name,
        "dni": dni,
        "birthdate": birthdate,
        "phone": phone,
        "location": location
    });
}
```

Figura 37- Código: Función para añadir pacientes

```
// Actualizar paciente
Future<void> updatePatient(String uid, String name, String birthdate,
    String phone, String location, String dni) async {
    await firestore.collection("patients").doc(uid).update({
        "name": name,
        "dni": dni,
        "birthdate": birthdate,
        "phone": phone,
        "location": location
    });
}
```

Figura 38- Código: Función para actualizar pacientes

```
//Eliminar paciente
Future<void> deletePatient(String uid) async {
  await firestore.collection("patients").doc(uid).delete();
}
```

Figura 39- Código: Función para eliminar pacientes

Esta serie de operaciones permiten que exista una relación directa entre la aplicación y la base de datos (Firestore Database) alojada en Firebase. Las instancias de las clases provenientes de librerías externas facilitan el flujo de información entre los cambios realizados en ambas plataformas.

Cabe destacar que esta serie de funciones se repiten de la misma manera en el caso de los doctores registrados en la aplicación. Esta gestión se personaliza teniendo en cuenta los atributos que deseemos añadir para caracterizar las diferencias.

8.1.2 Carga y análisis de documentos

Continuando con la estructura del capítulo, vamos a diferenciar este subapartado, correspondiente a una operación disponible en la app, de forma que quede claro cómo funciona internamente y el código que lo soporta.

Como bien sabemos este proceso consta de una serie de pasos, pero cuando el usuario pulsa el botón correspondiente a subir un archivo es el momento en el que necesitamos una función de código que gestione dicha subida.

```
class FirebaseStorageService {

  final FirebaseStorage storage = FirebaseStorage.instance;

  Future<String?> uploadFile(PlatformFile file) async {
    if (file.bytes == null) {
      return null; //Aseguramos que el archivo tiene datos
    }

    String fichero = '${DateTime.now().millisecondsSinceEpoch}_${file.name}';
    Reference referencia = storage.ref().child('reports/$fichero');

    try {
      await referencia.putData(file.bytes!);
      return await referencia.getDownloadURL();
    } catch (e) {
      print('Error al subir archivo: $e');
      return null;
    }
  }
}
```

Figura 40- Código: Función para la subida de archivos

Esta función asegura que el archivo no esté vacío antes de ser almacenado, genera un nombre único en base al timestamp actual para evitar colisiones entre elementos repetidos y lo sube a una carpeta creada 'reports' donde todos los informes quedarán en el historial.

En este caso, se ha optado por construir una clase en este script de servicio dedicado, esto no crea un cambio substancial con respecto a funciones independientes como se ha mostrado en imágenes anteriores, pero tiene unas pequeñas ventajas.

Básicamente las diferencias fundamentales radican en la organización, estructuración, legibilidad y escalabilidad. El código de esta agrupa funcionalidades comunes y reutiliza el uso de instancias, además permite añadir extensiones futuras.

Por esta parte, la subida de archivos quedaría completada, pero, como bien sabemos, esta característica de la aplicación viene ligada al correspondiente análisis de estos documentos almacenados por parte de la IA. Tras ser subidos, el contenido del documento se extrae y se realizan una serie de pasos hasta recibir una respuesta inteligente.

```
import 'package:cloud_functions/cloud_functions.dart';

class HuggingFaceService {
  final FirebaseFunctions functions = FirebaseFunctions.instance;

  Future<String> analyzeMedicalReport(String text) async {
    try {
      final HttpsCallable callable =
        functions.httpsCallable('analyzeMedicalReport');

      print("Enviando texto a Firebase Functions...");

      final respuesta = await callable.call({'text': text});

      //Se asegura de que response.data sea un Map antes de acceder a sus valores
      if (respuesta.data is Map && respuesta.data.containsKey('diagnosis')) {
        String diagnosis = respuesta.data['diagnosis'] as String;
        print("Diagnóstico recibido: $diagnosis");
        return diagnosis;
      }

      print("Error: Respuesta inesperada del servidor.");
      return "Error: Respuesta inesperada del servidor.";
    } catch (error) {
      print("Error al procesar el informe: $error");
      return "Error al procesar el informe. Por favor, inténtalo de nuevo más tarde.";
    }
  }
}
```

Figura 41- Código: Función para análisis de documentos

Una vez más se aprecia la estructura de 'clase', en este caso agrupamos la funcionalidad de la inteligencia artificial en este script que provee servicios en conjunto con HuggingFace.

Esta función de igual forma crea una instancia de Firebase Functions, lo que permite llamar a funciones en la nube desde Flutter. Con el método 'analyzeMedicalReport' se espera recibir el texto ingresado por el usuario y recibir una respuesta diagnóstica por parte de la IA.

Con la instrucción `httpsCallable('analyzeMedicalReport')` se indica que se llamará a una función en la nube de Firebase con ese nombre. Esta función está definida en otro script, `index.js`, que se mostrará y explicará a continuación.

Se llama a la función enviando el texto como parámetro y la función en Firebase Functions recibe el texto y lo reenvía a Hugging Face. La respuesta se imprime en la terminal y existe un manejo de errores que también muestran mensajes explicativos en caso de problemas con el servidor, o si las respuestas no son las esperadas.

El script `index.js` es más extenso y no se puede incluir una imagen completa de él, pero se explicarán sus funciones más representativas acompañadas de las líneas de código que las soportan:

```
const functions = require("firebase-functions");
const axios = require("axios");
const cors = require("cors");
const express = require("express");
```

Figura 42- Código: Importación de módulos en index.js

Primeramente, se añaden al script una serie de módulos, similares a librerías en otros scripts, que permiten hacer peticiones HTTP a Hugging Face y controlar el manejo de éstas, además de habilitar CORS.

```
const app = express();
app.use(cors({ origin: true }));
app.use(express.json());
```

Figura 43- Código: Configuración del servidor en index.js

Estas líneas de código permiten recibir desde cualquier origen y manejar JSON.

```
exports.analyzeMedicalReport = functions.https.onCall(async (data, context) => {
  console.log("Solicitud recibida:", data);

  let texto = data?.data?.text;
  if (!texto || typeof texto !== "string") {
    throw new functions.https.HttpsError("invalid-argument", "El texto es obligatorio y debe ser una cadena.");
  }
});
```

Figura 44- Código: Creación de la función en index.js

Aquí vemos cómo comienza a construirse la función que se definía en la nube de Firebase y que se ejecutará cuando Flutter la llame. También aparece una validación de datos, esto es para asegurarnos de que se reciba una cadena manejable.

```
const mensaje = texto.trim() + " ¿Diagnóstico?";
```

Figura 45- Código: Instrucción a enviar desde index.js

Esto añade al final del input la pregunta: ¿Diagnóstico? Esto se hace para que la IA interprete correctamente que tiene que generar un diagnóstico para los síntomas expresados por el paciente.

```
try {
  const huggingFaceUrl = "https://api-inference.huggingface.co/models/mistralai/Mistral-8x7B-Instruct-v0.1";
  console.log("Enviando texto a Hugging Face:", prompt);

  const respuesta = await axios.post(
    huggingFaceUrl,
    {
      inputs: prompt,
      parameters: {
        max_length: 2048,
        temperature: 0.5
      }
    },
    {
      headers: {
        Authorization: `Bearer hf_HBWZpOSxGKVWHiVxlyFstEerzAnxUqZiW`,
        "Content-Type": "application/json"
      }
    }
  );
}
```

Figura 46- Código: Petición a HuggingFace en index.js

Podríamos decir que este es el ‘core’ del script, ya que representa la parte más esencial de este. Como se puede observar, estamos almacenando la URL de nuestro modelo de IA de HuggingFace en una variable para poder usarla posteriormente.

Además, aquí se construye la petición en formato JSON, con el respectivo ‘input’, los caracteres que pedimos como máximo en la respuesta (2048), la temperatura (0.5), que es un parámetro que maneja la creatividad, precisión y aleatoriedad de las respuestas de la IA y nuestro token de acceso que necesitamos para acceder a las funciones de este modelo.

```
let diagnosis = "No se obtuvo diagnóstico.";
if (Array.isArray(respuesta.data) && respuesta.data.length > 0 && respuesta.data[0].generated_text) {
  diagnosis = respuesta.data[0].generated_text.trim();
}

//Eliminar la pregunta inicial si está repetida en la respuesta
if (diagnosis.startsWith(prompt)) {
  diagnosis = diagnosis.replace(prompt, "").trim();
}

console.log("Diagnóstico generado:\n" + diagnosis);
return { diagnosis };
```

Figura 47- Código: Procesamiento de la respuesta en index.js

En este apartado se declaran variables por defecto en caso de errores, se elimina el 'prompt' de la respuesta para evitar duplicidad innecesaria, se registran comentarios en el log y se devuelve la respuesta.

```
} catch (error) {  
  console.error("Error procesando el informe:", error.respuesta?.data || error.message);  
  throw new functions.https.HttpsError("internal", "Error al procesar el informe con Hugging Face.");  
}
```

Figura 48- Código: Control de errores en index.js

Por último, aparece este control de errores correspondiente a la estructura 'try-catch' típica de programación para el manejo de posibles errores al realizar peticiones o realizar peticiones con posible lanzamiento de excepciones. Se registran errores en la consola y se lanzan para manejarlos en el frontend.

8.1.3 Conversión de voz a texto

Para el siguiente servicio prestado por el sistema, se informará de los aspectos internos de los que está compuesta esta funcionalidad. La conversión de voz a texto en la aplicación permite a los usuarios registrar información médica de manera más natural mediante el reconocimiento de voz. Esta característica se implementa a partir de librerías de reconocimiento y transcripción de audio.

En las posteriores imágenes se hará un seguimiento del script implicado en realizar esta tarea, con las explicaciones necesarias de las diferentes partes y funciones que componen el código.

```
import 'package:cloud_firestore/cloud_firestore.dart';  
import 'package:firebase_auth/firebase_auth.dart';  
import 'package:flutter/material.dart';  
import 'package:speech_to_text/speech_to_text.dart' as stt;  
import 'package:crud_firebase/services/huggingface_service.dart';
```

Figura 49- Código: Librerías necesarias en Voz-Texto

Como se ha explicado anteriormente, las librerías son de vital importancia para el uso de funciones externas. En este caso, se hace uso de una instancia de una librería, que estará presente a lo largo del código.

```
class VoiceService extends ChangeNotifier {  
  final stt.SpeechToText _speech = stt.SpeechToText();  
  bool _escuchando = false;  
  final TextEditingController controlador = TextEditingController();  
  
  bool get escuchando => _escuchando;
```

Figura 50- Código: Definición de clase y variables Voz-Texto

En estas primeras líneas tan sólo aparecen algunas definiciones. La clase extiende `ChangeNotifier` para permitir la actualización de la interfaz cuando cambien sus variables internas. Las demás variables gestionan la conversión de voz a texto, indican si el sistema está en modo de escucha y controlan el texto transcrito.

```
Future<void> toggleListening() async {  
  if (_escuchando) {  
    stopListening();  
  } else {  
    await startListening();  
  }  
}
```

Figura 51- Código: Función para alternar escucha Voz-Texto

Aquí se presenta la primera función del script, haciendo uso de variables anteriores, sentencias condicionales y funciones que serán explicadas posteriormente. El hecho de que esta función y las siguientes sean asíncronas permiten que el programa siga ejecutándose mientras esperan resultados y no se bloqueen en flujo.

```
Future<void> startListening() async {  
  bool disponible = await _speech.initialize(  
    onStatus: (status) => print("Estado: $status"),  
    onError: (error) => print("Error: $error"),  
  );  
  
  if (disponible) {  
    _escuchando = true;  
    _speech.listen(  
      onResult: (result) {  
        controlador.text = result.recognizedWords;  
        notifyListeners();  
      },  
      localeId: "es_ES",  
    );  
    notifyListeners();  
  } else {  
    print("Reconocimiento de voz no disponible.");  
  }  
}
```

Figura 52- Código: Función de escucha Voz-Texto

Esta es la función para iniciar la escucha tras pulsar el respectivo botón en el interfaz de la aplicación.

Si el reconocimiento está disponible, empieza a escuchar, guarda las palabras reconocidas en el controlador de texto y establece los tiempos máximos de escucha y pausas. A su vez, también gestiona eventos de estado y errores.

```
void stopListening() {  
  _escuchando = false;  
  _speech.stop();  
  notifyListeners();  
}  
  
void resetText() {  
  controlador.clear();  
  notifyListeners();  
}
```

Figura 53- Código: Detención y Reset Voz-Texto

Estas 2 funciones tienen una implementación más simple gracias a las variables y funciones mostradas antes. Aquí hay un cambio de valor en la variable para detener la escucha y se elimina el contenido del controlador.

```
Future<void> sendToIA(String patientName, String patientId) async {
  try {
    final usuario = FirebaseAuth.instance.currentUser;
    if (usuario == null || controlador.text.isEmpty) return;

    //Aquí usamos la función de análisis de documentos
    String diagnosis =
      await HuggingFaceService().analyzeMedicalReport(controlador.text);

    //Guardamos en Firestore
    await FirebaseFirestore.instance.collection('transcriptions').add({
      'patientName': patientName,
      'userID': usuario.uid,
      'timestamp': FieldValue.serverTimestamp(),
      'problem': controlador.text,
      'diagnosis': diagnosis,
    });

    print("Transcripción guardada en Firestore con diagnóstico.");
    resetText();
  } catch (e) {
    print("Error guardando en Firestore: $e");
  }
}
```

Figura 54- Código: Envío a IA Voz-Texto

Además de lo comentado anteriormente, este código tiene una función específica de envío de información a la IA. Como lo dicho en apartados anteriores, en esta parte de la aplicación también se desempeña un análisis médico de la inteligencia artificial. El texto extraído de la voz del paciente es enviado para ser analizado, esto es posible gracias al uso de funciones de otros scripts de otros apartados de esta sección.

En esta ocasión, se hace un almacenamiento en la base de datos con la siguiente información: Datos del paciente, usuario administrador, fecha actual, el problema que se ha identificado y el diagnóstico generado para dicho problema. Todo queda almacenado en formato de documento y a su vez en una colección de transcripciones. De esta forma, se obtiene un registro del historial médico ordenado y accesible para futuras consultas.

8.2 Decoración

El punto anterior intentaba demostrar la estructura interna y el funcionamiento de la aplicación desde su código Dart, enumerando los distintos servicios, navegación y utilidades que la componen.

En este apartado se tratará la decoración. El código de la app no sólo contiene instrucciones que facilitan el flujo de información y el funcionamiento, sino que también presenta componentes visuales dedicadas a la estructuración y orden gráfica de la interfaz. Gracias a todo lo que engloba este punto, el sistema muestra una IU sencilla, vistosa y atractiva hacia el cliente o usuario final, habilitando un uso fácil e interactivo.

8.2.1 Principios de diseño

La aplicación sigue las directrices de un sistema de diseño desarrollado por Google para garantizar una experiencia visual uniforme en las terminales móviles y web. Estamos hablando de **Material Design**, que sigue estos principios:

- Uso de colores adecuados y transiciones suaves para una buena experiencia.
- Jerarquía visual clara con distintos niveles de profundidad.
- Animaciones y respuestas interactivas que mejoren la experiencia del usuario.

En Flutter, Material Design se implementa a través de widgets predefinidos como 'Scaffold', 'AppBar', 'ElevatedButton', 'TextField', y otros. La mayor parte de los componentes gráficos de esta aplicación se han construido utilizando estos elementos.

8.2.2 Estructura General de los Widgets

El diseño de cada pantalla se basa en una jerarquía de widgets organizada de manera estructurada. Todas las pantallas siguen una arquitectura similar basada en el uso del método 'build', que define la estructura de cada interfaz de usuario.

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text("Gestión de pérdida de credenciales"),
      backgroundColor: Colors.purple.shade700,
      centerTitle: true,
    ), // AppBar
    body: Container(
      decoration: BoxDecoration(
        gradient: LinearGradient(
          colors: [
            Colors.purple.shade700,
            Colors.purple.shade500,
            Colors.purple.shade300,
          ],
          begin: Alignment.topCenter,
          end: Alignment.bottomCenter,
        ), // LinearGradient
      ), // BoxDecoration
    ),
```

Figura 55- Código: Estructura de diseño geneneralizada

La imagen anterior corresponde a un esquema general de cómo se compone este método a lo largo de las diferentes páginas propuestas que conforman la aplicación. Cabe resaltar que este fragmento de código es tan sólo ilustrativo a modo de ejemplo en el que se ha intentado mostrar una estructura general para todos.

En este caso se ha usado un degradado de tonos morados y se han incluido algunos textos y elementos decorativos.

Cada página principal utiliza un widget 'Scaffold', que proporciona la base de la pantalla e incluye elementos como:

- Un 'AppBar' para la cabecera en algunas secciones.
- Un 'Container' con decoración para establecer el fondo y aplicar degradados.
- Un 'Column' o 'ListView' para organizar los elementos visuales de forma vertical.
- Widgets específicos (TextField, ElevatedButton, Icon, etc.).

Gracias a estos widgets y la combinación de los mismos es posible crear diferentes pantallas creativas y originales que sean de agrado para el receptor.

Capítulo 9. Pruebas y Validación

Para este apartado de pruebas, se realizaron una serie de comprobaciones intentando abarcar diferentes tipos de verificación para que el sistema completo se ejecute con éxito. Esto incluye que todos los requisitos funcionales estén validados y que el flujo de datos existente esté libre de errores. Para lograr esto, se han necesitado varios tipos de pruebas.

9.1 Pruebas funcionales

Las pruebas funcionales en sí mismas son las encargadas de que todos y cada uno de los aspectos o servicios que ofrece el sistema funcionen por separado, es decir, que reaccionen como se espera que lo hagan.

Realmente, durante la construcción del sistema, se procuró realizar la implementación en base a capas seguras. Esto quiere decir que todas las funcionalidades que se intentaban ofrecer en la aplicación eran comprobadas antes de añadir otras nuevas, de esta forma se iba formando una base sólida e hizo más fácil que el sistema global tuviese menos errores.

Se realizaron pruebas individuales para cada módulo de la aplicación, asegurando que cada componente cumpliera con su propósito dentro del flujo de trabajo.

9.1.1 Autenticación del usuario

Para acceder a la aplicación, el usuario que la controla debe autenticarse mediante un sistema de credenciales basado en correo y contraseña previamente registrados en Firebase Authentication, para que el control y uso de esta sea para personal autorizado, prevenga posibles usos fraudulentos, introducción de datos falsos en las bases de datos, modificación de información personal y cualquier tipo de malware asociado. Se realizaron las siguientes pruebas:

- ✓ Ingreso con credenciales correctas: Se verificó que un usuario con credenciales válidas pueda acceder sin inconvenientes.
- ✓ Ingreso con credenciales incorrectas: Se probó que el sistema muestre un mensaje de error y no permita el acceso.
- ✓ Campos de texto disponibles: Se aseguró que el usuario pudiese escribir de teclado sus credenciales en los espacios habilitados.
- ✓ Cierre de sesión: Se comprobó que al concluir el uso vigente se exigiesen de nuevo las credenciales para un nuevo ingreso.
- ✓ Recuperación de contraseña exitosa: Se probó a efectuar un cambio de contraseña para comprobar la eficacia del método.

9.1.2 Gestión CRUD de Médicos y Pacientes

El administrador tiene el poder o capacidad de agregar, modificar o eliminar los usuarios registrados en la base de datos y hacer que estos estén presentes en la interfaz de usuario de la aplicación en tiempo real. Se realizaron las siguientes comprobaciones:

- ✓ Creación de nuevos registros: Se validó que el administrador pudiese agregar médicos y pacientes rellendo cada uno de los campos o atributos que le pertenecen a cada uno de ellos.
- ✓ Edición de registros existentes: Se comprobó que la actualización de datos en la interfaz se refleje correctamente en la base de datos.
- ✓ Eliminación de registros: Se observó que los cambios realizados en cuanto a la terminación de algunos usuarios se actualizasen en tiempo real.
- ✓ Bilateralidad: Se aseguró que tanto los cambios provenientes desde la IU como la propia modificación de colecciones desde la base de datos en sí, se viese reflejada en ambas pantallas.
- ✓ Tiempo real: Se trabajó en que dichos cambios fuesen observables instantáneamente sin necesidad de refrescar las webs o reiniciar la aplicación.

9.1.3 Acceso a enlaces externos

La 'Home Page' de la app tiene algunos enlaces a webs externas relacionadas con la salud para tener disponibilidad directa a información concreta o investigar acerca de necesidad en entidades superiores. Se aseguró lo siguiente:

- ✓ Redireccionamiento adecuado: Se verificó que al hacer clic en los enlaces se abra la página correspondiente en un navegador.
- ✓ Disponibilidad: Se revisó que los enlaces estuvieran activos y no llevaran a páginas inexistentes.

9.1.4 Análisis de informes

Una de las utilidades principales es poder subir texto médico a través de la interfaz de la aplicación para que este pueda ser analizado por la inteligencia artificial, la cual devolverá un diagnóstico identificado para los problemas presentados. Además, dicho informe quedará guardado en Firebase Storage, en una carpeta con todos los archivos que se suban. Se hicieron las siguientes pruebas:

- ✓ Interfaz interactiva: Se procuró que el botón presente en la pestaña de la app, pudiese ser pulsado y abriese el explorador de archivos del dispositivo.
- ✓ Selección adecuada: Se verificó que el formato de archivo disponible a seleccionar fuese el aceptado por todos los módulos.
- ✓ Almacenamiento directo: Se comprobó que, al aceptar la selección de un archivo con texto, este se almacenara con éxito en la carpeta correspondiente.

- ✓ Respuesta asegurada: Se garantizó que, al subir el texto médico, se recibiera una respuesta acorde con el problema propuesto.
- ✓ Información disponible: Se observó que aparecían mensajes explicativos en la interfaz para notificar el éxito o el fallo de la operación.

9.1.5 Transcripción de voz a texto

El sistema permite asignar este tipo de servicio de conversión a una de las personas registradas como paciente en la base de datos. Tras su selección, se presta la oportunidad de que dicho usuario pueda expresar su estado de salud para ser analizado a posteriori. Para que este servicio se considerara fiable, se tuvieron en cuenta los siguientes aspectos:

- ✓ Selección actualizada: Se comprobó que se pudiese elegir entre los pacientes existentes registrados para dedicar un servicio personal.
- ✓ Botón de grabación: Se testeó el pulsador de inicio de escucha de audio funcionase y se iniciara la transcripción.
- ✓ Edición garantizada: Se aseguró que el texto transcrito fuera manualmente modificable para corregir cualquier error cometido.
- ✓ Diagnóstico: Se revisó que al enviar el texto concreto se recibiese una respuesta en relación con el problema presentado.

9.2 Pruebas de integración

Por otro lado, este tipo de pruebas integrales se basan mayoritariamente en la conexión fluida de todos y cada uno de los requisitos funcionales detallados con anterioridad. Es decir, en lugar de tener en cuenta que cada elemento funcione aisladamente, se busca un trabajo conjunto de la aplicación completa.

En el caso de esta aplicación, dichas pruebas se centran en comprobar la correcta interacción entre el sistema de autenticación, la base de datos, la carga y análisis de documentos, la transcripción por voz y la generación de diagnósticos. Tienen como objetivo validar que los diferentes módulos de la aplicación colaboran correctamente entre sí, reproduciendo escenarios reales de uso.

▪ Autenticación + Acceso al sistema de gestión:

El objetivo principal se basa en que, tras un login exitoso el usuario administrador pueda acceder a las funciones de gestión de la aplicación. Al introducir las credenciales de forma correcta, el sistema reacciona de tal forma que identifica que el usuario es válido para acceder a las funciones que ofrece, rediriéndolo a la página principal.

Una vez dentro, el usuario dispone de total libertad para moverse entre las pestañas de navegación disponibles, esto asegura que el sistema de autenticación es correcto y que la redirección fue exitosa.

- Navegación + acceso a enlaces externos

Confirmar que los enlaces a recursos externos de salud funcionan correctamente desde la aplicación. El procedimiento que se siguió fue, una vez situados en la página principal de navegación de la app, hacer clic en los diferentes enlaces propuestos, esperando una redirección adecuada y disponible para seguir navegando en webs externas sin fallo y con la libertad de volver a la aplicación con total seguridad.

Los módulos se comunican con éxito y la flexibilidad entre ventanas es la esperada.

- Gestión CRUD + Actualización de interfaz en tiempo real

Asimismo, con el siguiente apartado se trató de validar que los cambios introducidos fueran correctamente actualizados en la interfaz de usuario. Para llevar esto a cabo, se necesitó realizar pruebas específicas para pacientes y médicos, añadiendo nuevos, modificando atributos, eliminando desde los iconos accesibles y desde la propia base de datos, y asegurar los cambios formados en cada ventana.

Gracias a esto es posible comprobar que cada uno de los requisitos individuales que exigía este apartado forman un conjunto accesible, disponible y sin errores que funciona para todas las combinaciones.

- Carga de documentos + Generación de diagnóstico

Los aspectos independientes del análisis de texto médico son claros, pero era necesario verificar que los pasos por los que transcurre la aplicación se realizan de forma correcta y ordenada, reenviando la información a los módulos adecuados y recibiendo información de vuelta con sentido y en un límite de tiempo aceptable.

Sabemos que al cargar texto debemos recibir una respuesta, así que se probó manualmente con archivos de prueba para asegurar que la información gestionada por Firebase Functions y HuggingFace seguía un esquema coherente y se devolvía la información que se esperaba recibir.

- Asociación al paciente + Transcripción de voz + Generación de diagnóstico + Actualización en la base de datos

Por último y de igual forma, el último apartado también necesitaba una serie de pruebas de integración, ya que es el que más pasos le corresponde, el que más información maneja y con el que intervienen todos los módulos.

La navegación hasta la ventana de selección se proporcionaba suavemente y sin inconvenientes, el desplegable funcionaba a la perfección mostrando la variedad de pacientes presentes en la base de datos. Una vez elegido, la interfaz de usuario cambiaba sin errores, proporcionando un entorno sencillo para probar las funcionalidades.

Al iniciar la transcripción de voz, el navegador pregunta por los permisos de micrófono y, al aceptar, se procede a realizar una conversión textual de las palabras sonadas en voz alta. La interfaz interactiva también funcionaba, haciendo el campo de texto tuviera capacidad de edición y aceptando palabras escritas de teclado.

Para terminar, la funcionalidad añadida de análisis funcionaba de la misma forma que la pestaña de almacenamiento de documentos con los atributos personalizados y comunicándose con los demás servicios externos sin inconvenientes.

9.3 Posibles mejoras detectadas

En este punto del capítulo ya no sólo hablamos del trabajo que ha sido desempeñado durante el desarrollo de la aplicación, sino que pasamos a un momento más reflexivo y crítico acerca de aspectos mejorables del proyecto.

Honestamente creo que encontrar la perfección es algo complicado por no decir imposible ya que eso no sólo depende de tu propio juicio sino de personas ajenas que valoran tu trabajo. Por eso mismo, creo que siempre hay lugar para mejorar y en este trabajo no lo es menos. Al realizar un análisis exhaustivo, se deja la puerta abierta para futuras versiones del proyecto o incluso servir de punto de partida para otros desarrolladores.

Durante el desarrollo, implementación y pruebas de la aplicación, se identificaron una serie de aspectos que podrían mejorarse o ampliarse en futuras versiones. A continuación, se presentan las mejoras más relevantes:

1. Mejora de la seguridad y control de acceso

Como bien sabemos la aplicación cuenta con un sistema de autenticación mediante email y contraseña para el usuario administrador.

Por una parte, se podrían elegir métodos de verificación más sofisticados, hoy en día las aplicaciones con mayor seguridad cuentan con un servicio de acceso mediante credenciales biométricas, esto quiere decir que están adaptadas a características físicas personales de cada usuario y que son intransferibles.

En adición, otra mejora propuesta para el control de la aplicación podría ser un uso dedicado.

Con esto me refiero a la implementación de un sistema de roles en el que, dependiendo del tipo de persona que acceda a la aplicación, se presten unas necesidades otras según los requerimientos del usuario.

En parte esto se ha desarrollado de una forma primitiva, intentando diferenciar el uso de la app por parte de un médico o un administrador, pero esta posible mejora detectada se refiere a una ampliación mayor, probablemente personalizando la interfaz y funcionamiento para distintos departamentos médicos.

2. Resiliencia ante errores en el modelo de IA

En la situación actual de la aplicación se depende de una API externa que colabora para integrar sus servicios de forma que queden conectados de manera uniforme y cada vez que se solicite.

El problema que esto presenta es que, si el servicio se interrumpe, o si no se disponen de los tokens necesarios para realizar estas peticiones, la funcionalidad se ve afectada.

Para mejorar esto, se podría implementar un sistema de gestión de errores más robusto que informe al usuario de forma clara y que ofrezca alternativas, como un reintento, una cola de espera para el procesamiento o mensajes personalizados si la IA no responde.

3. Historial de usuarios más detallado

Lo que viene a referir este apartado es un aumento en los atributos y características que presentan los usuarios que conforman la base de datos. Hasta el momento en la aplicación no se han introducido una cantidad de campos elevada en las colecciones para facilitar las pruebas y no hacer el proceso demasiado tedioso.

En casos profesionales, las personas poseen variedad de información disponible para diferenciarse del resto. La introducción de esta información extra no supondría mayor complicación, aunque probablemente se llegaría a un punto en el que fuese necesario un filtrado.

Si la cantidad de pacientes o médicos aumenta considerablemente, se llegará un punto en el que se tengan muchos con atribuciones parecidas y para diferenciar entre unos y otros y realizar búsquedas más rápidas se necesitarían mecanismos de descarte.

4. Mejora en la experiencia de usuario

En mi opinión en este apartado considero que existe un rango de mejora infinito, como dije anteriormente, cada usuario tiene sus gustos propios y es muy difícil encontrar una interfaz que sea vistosa y agradable para todos.

A pesar de haber incluido colores degradados, títulos, imágenes, iconos, recuadros de texto, ventanas de navegación, elementos intuitivos, transiciones y demás, creo que se podrían añadir muchos más aspectos para mejorar la interfaz del usuario. Esto podría hacerse con mayor uso de librerías externas, implementaciones novedosas y divertidas y mucha creatividad.

Para satisfacer necesidades externas, sería necesario que usuarios externos probasen la aplicación y colaborasen dando feedback según su opinión y posibles mejoras futuras que ellos incorporarían.

5. Adaptabilidad a múltiples dispositivos

La aplicación está diseñada con responsive básico, pero no se ha adaptado específicamente a tablets o pantallas grandes. Una mejora propuesta sería introducir lógica de diseño adaptable para optimizar la visualización en distintos formatos.

Además de esto, este sistema fue pensado y desarrollado para soporte web y poder ser lanzado en algunos navegadores. En este punto se podría extender el lanzamiento y soporte de esta aplicación en mayor amplitud de navegadores o plataformas móviles.

Se evalúa la posibilidad de versiones de escritorio y de extensión a otro tipo de plataformas.

6. Optimización del sistema de carga y almacenamiento de archivos

Actualmente, esta pestaña de análisis de documentos sólo acepta aquellos que estén en formato texto para posteriormente ser analizados. Una mejora evidente sería añadir mayor variabilidad de extensiones de archivos disponibles, pudiendo disfrutar de una carga de archivos en formato pdf o docx entre otros.

Por consiguiente, durante el proceso de almacenamiento se podrían añadir barras de carga mostrando porcentajes en la subida además del mensaje de éxito que ya está disponible. De otra forma, los archivos podrían clasificarse en diferentes carpetas según distintas necesidades o filtros implicados y mejorar la gestión del almacenamiento en Firebase para evitar acumulación innecesaria.

7. Mejoras en el sistema de reconocimiento de voz

El sistema convierte audio a texto para generar diagnósticos personalizados, pero de momento sólo entiende el castellano y el inglés como idiomas de reconocimiento.

Como extensión, sería una buena idea introducir mayor variedad de idiomas disponibles, para que diferentes usuarios pudiesen gozar de los servicios con su lengua nativa.

Capítulo 10. Conclusiones y Trabajo Futuro

A lo largo de este trabajo se ha desarrollado una aplicación que integra distintas tecnologías con el propósito de asistir en procesos médicos mediante la inteligencia artificial. Durante todo el desarrollo de la memoria se han ido recalcando las funcionalidades, herramientas y módulos que posee la aplicación.

Sin embargo, para este capítulo del trabajo me gustaría, además de hacer un resumen completo de todos los requisitos cumplidos, dar un poco de contexto, profundidad y valor añadido. Con el desarrollo de este tipo de aplicaciones estamos conectando la tecnología con entornos reales de aplicación en la sociedad de hoy en día.

Quisiera mostrar conciencia del desarrollo al tener en cuenta como estamos adaptando herramientas tan capaces como la IA, en ámbitos de aplicación tan fundamentales como lo es la medicina.

Haciendo especial mención a la inteligencia artificial, me he dado cuenta de que en un período relativamente corto de tiempo hemos creado una herramienta inimaginablemente poderosa. Y es que este tipo de conocimiento y control de los ordenadores no sólo ayuda a mejorar la eficiencia de ámbitos cotidianos o reiterativos como las consultas médicas, sino que intervienen en muchísimas operaciones de alta complejidad y lo hacen de una manera óptima.

Este tipo de tecnología es tan eficiente que está en continuo aprendizaje y aseguramos que es vertiginosamente más rápido que el que un ser humano podría llegar a plantearse. A consecuencia de esto, nadie sabe lo que nos deparará de aquí a un año, cinco o diez, pero lo que sí sabemos es que la IA está optimizando muchos sectores, provocando desaparición de empleo en numerosos sectores en los momentos que transcurren.

10.1 Resumen de los logros alcanzados

Por si cupiera lugar a dudas, este apartado del proyecto se centra en repasar todos aquellos aspectos que hemos tratado a lo largo de la memoria, de los objetivos que hemos cumplido construyendo este sistema y haciendo un análisis cualitativo del trabajo desempeñado.

Primeramente, me gusta pensar que el objetivo inicial que propusimos al inicio del trabajo está alcanzado. Y es que, en mayor o menor medida, creo que he podido crear una herramienta que puede servir de ayuda en cuanto a la mejora de la eficiencia en las consultas médicas. Al fin y al cabo, nuestra meta era cumplir con el avance de esta tecnología en la medicina y con esta aplicación hacemos una propuesta considerable para ofrecer valor real en contextos sensibles e importantes como la salud.

Sin irnos más lejos, hemos desarrollado una aplicación totalmente funcional y accesible con distintos módulos integrados que colaboran entre sí para ofrecer un servicio confiable, rápido, eficiente y en tiempo real. Si enumeramos las capacidades desempeñadas por esta aplicación:

- La integración de un sistema de autenticación seguro mediante correo electrónico y contraseña, permitiendo el acceso múltiple.
- El diseño e implementación de un sistema completo de gestión de pacientes y médicos, con funcionalidades CRUD en tiempo real.
- El almacenamiento estructurado y accesible de todos los datos generados en una base de datos en la nube.
- El uso de reconocimiento de voz para transformar conversaciones en texto, lo que simula entornos clínicos reales.
- La conexión con modelos de IA capaces de generar diagnósticos o análisis basados en los síntomas descritos ya sea por voz o desde documentos médicos.
- La creación de un formato de conversación estilo chat para continuar consultas dedicadas o abarcar nuevas enfermedades.
- La creación de una interfaz limpia y enfocada a la experiencia del usuario, usando principios del 'Material Design'.

10.2 Limitaciones y mejoras para futuras versiones

Más allá de los aspectos técnicos, este trabajo ha permitido visualizar el papel que puede jugar la inteligencia artificial en el futuro de la medicina. Esta aplicación representa un ejemplo funcional, aunque primitivo, de cómo estas tecnologías pueden colaborar con profesionales sanitarios, optimizando tiempos de respuesta, mejorando el acceso a información y facilitando la toma de decisiones clínicas.

Aunque se ha conseguido buena funcionalidad, es importante tener en cuenta que las aplicaciones médicas impulsadas por IA requieren un alto grado de precisión y responsabilidad, ya que se trata de nada menos que vidas humanas.

Algunas limitaciones actuales de esta aplicación son aspectos que deben abordarse en futuros desarrollos si se quisiera escalar la solución a un entorno clínico real. Como dije en capítulos atrás, siempre hay lugar para la mejora y me gustaría averiguar qué tan lejos se puede llegar mejorando y añadiendo versiones a esta línea de trabajo. Se me ocurren futuras líneas de mejora con enfoques como:

- Colaboración con profesionales médicos reales que pudiesen aportar retroalimentación acerca de las decisiones, validar los resultados y mejorar la calidad de los diagnósticos generados.

- Entreno más especializado de la IA, con entrenamientos sobre bases de datos clínicos concretos y con mecanismos de explicación.
- Apertura del sistema a distintos roles, pensamiento que ya se dio anteriormente en el apartado 9.3 de la memoria.
- Cambio y edición de enlaces existentes para abrir puertas a nuevas conexiones y redirecciones.
- Posible consideración de la ética en la toma de decisiones en algunas situaciones clínicas complejas. Todos nos hemos puesto en situación de momentos complicados en los que interviene la ética en la toma de decisiones médicas, la IA actuaría de manera imparcial sin tener en cuenta el apartado sentimental y valorando las mejores opciones para el afectado. De esta forma no sólo se involucraría en medicina, sino también en jurisdicción.
- Internacionalización y accesibilidad, para llegar a contextos donde la asistencia médica es limitada y donde este tipo de herramientas pueden suponer una diferencia significativa.
- Limitación del uso por parte de la IA. Como comentábamos en el apartado de tecnologías utilizadas, la API de HuggingFace no ofrece uso ilimitado y dado que la cantidad de peticiones HTTP permitidas por parte de la aplicación es ínfima a gran escala, se debería tener en cuenta el costo extra que supone la suscripción Pro de esta herramienta.

Si bien el objetivo inicial era académico, lo que aquí se ha construido puede abrir la puerta a nuevas soluciones tecnológicas en salud digital, atención primaria asistida por IA, y telemedicina inteligente.

Capítulo 11. Manual de Instalación

El siguiente manual de instalación se utilizará a modo de guía rápida para la puesta en marcha de este proyecto en un dispositivo externo al que ha sido desarrollado. Durante los próximos apartados, se proporcionará información acerca de los requerimientos necesarios y pasos de instalación.

11.1 Requerimientos

Antes de proceder con la instalación, se han de tener las siguientes herramientas para asegurarnos de que no se dará paso a errores en el proceso:

- ✓ Sistema operativo Windows 11
- ✓ Flutter SDK (versión 3 en adelante)
- ✓ VS Code (con las extensiones de Flutter y Dart)
- ✓ Cuenta de Firebase con un proyecto en blanco
- ✓ Cuenta de HuggingFace con un Access Token y permisos habilitados.

11.2 Pasos de instalación

Este apartado se dedica a explicar con el mayor grado de detalle posible, los pasos que serían necesarios para replicar el uso de esta aplicación en otros dispositivos.

Paso 1: Clonar o almacenar el proyecto

En este primer paso se procura que el usuario que pretende replicar el proyecto tenga los mismos archivos de código ejecutándose en su máquina. Para ello puede hacer uso de la herramienta Git con los siguientes comandos:

```
git clone <https://github.com/ItsPabloGR-1/TFG>  
cd nombre-del-proyecto
```

O simplemente copiar la carpeta con todos los archivos en su máquina local.

Paso 2: Instalación de herramientas

Para aplicar este proyecto, es necesaria la instalación de los requerimientos presentes en el apartado 11.1 de este capítulo. Puedes verificar que Flutter está correctamente instalando con el comando:

```
flutter doctor
```

En caso de que apareciesen algunos impedimentos, la terminal ofrece una guía resolutive de cómo cumplir con las características restantes y completar con la instalación al completo. Deberás ver algo como lo siguiente:

```
PS C:\WINDOWS\system32> flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.27.4, on Microsoft Windows [Versi n 10.0.26100.3775], locale es-ES)
[✓] Windows Version (Installed version of Windows is version 10 or higher)
[✓] Android toolchain - develop for Android devices (Android SDK version 35.0.1)
[✓] Chrome - develop for the web
[✓] Visual Studio - develop Windows apps (Visual Studio Community 2022 17.12.1)
[✓] Android Studio (version 2024.2)
[✓] VS Code (version 1.100.0)
[✓] Connected device (3 available)
[✓] Network resources

• No issues found!
```

Figura 56- Pasos de instalaci n: 'flutter doctor'

Paso 3: Configurar Firebase

Para tener acceso a los numerosos servicios que nos ha prestado esta herramienta, ser  necesario darse de alta con una cuenta v lida y crear un proyecto en Firebase Console.

(<https://console.firebase.google.com/>)

- Activa Firebase Authentication y crea unas credenciales de entrada.
- Configura el Cloud Firestore para dar paso al uso de una base de datos en modo prueba.
- Habilita Firebase Storage para el almacenamiento de documentos m dicos subidos desde la app.
- Por  ltimo, descarga el archivo google-services.json y col calo en el directorio android/app de tu proyecto.

Paso 4: Instalar las dependencias del proyecto

Dentro del directorio del proyecto, debes abrir una terminal y lanzar el siguiente comando:

flutter pub get

Esto har  que se descarguen todas las bibliotecas necesarias existentes en el archivo pubspec.yaml

Paso 5: Firebase Functions

Aseg rate de tener Node.js instalado, puedes verificarlo con:

node -v

Posteriormente, inicializa esta herramienta en tu proyecto a través de este otro comando:

firebase init functions

Dado que el código maneja las funciones que tratarán con las solicitudes de la API de HuggingFace, no es necesario que añadas nada más.

Paso 6: HuggingFace

Necesitarás crear una cuenta de este repositorio web para acceder a sus servicios, puedes hacerlo de forma rápida a través del método de inicio de sesión que ofrece con Google a través de tu propia cuenta.

Una vez logueado, deberás crear tu propio Token de acceso para conectarte con el modelo de IA. Para ello, navega hasta el apartado de 'Acces Tokens' y pulsa el siguiente botón:

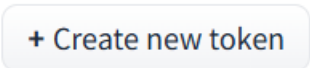


Figura 57- Pasos instalación: 'Create new token'

Será obligatorio nombrar a este token de conexión de la forma que consideres más oportuna y no se puede olvidar seleccionar la opción de 'Inference Providers', te lo muestro en la siguiente imagen:

Inference



Make calls to Inference Providers



Make calls to your Inference Endpoints ⓘ



Manage your Inference Endpoints ⓘ

Figura 58- Pasos de instalacion: Permiso de 'Inference Providers'

Antes de cerrar la pestaña, asegúrate de copiar y almacenar el token que te ha sido generado, dado que no podrás volver a visualizarlo a través de esa interfaz (hay un mensaje de aviso autogenerado).

Sustituye tu propio token de acceso en el apartado de 'headers' del archivo index.js del proyecto, que es el momento en el que se genera la solicitud de conexión.

Paso 7: Ejecución del proyecto

Una vez todo esté preparado y tras haber cerciorado que no hay errores existentes, tendrás todo listo para probar el comando de ejecución en la terminal.

flutter run

Capítulo 12. Manual de Usuario

Para el último capítulo redactado de la memoria se propondrá un manual de usuario extendido sobre el uso de la aplicación. A lo largo de este, se mostrarán capturas directamente recogidas de la interfaz de usuario para hacer una alusión a cada ventana disponible y sus funcionalidades implementadas.

12.1 Inicio de Sesión

En las Figuras 60 y 61 se mostrará la pantalla inicial que aparece tras el lanzamiento de la aplicación. Como bien sabemos, se trata de una ventana inicial de autenticación, en la que aparecen elementos decorativos, imágenes, colores, botones, títulos y recuadros habilitados para el reconocimiento de credenciales. Además, se muestra un caso de autenticación fallida, en el que los datos introducidos no son válidos para habilitar el uso de la aplicación.

La imagen muestra la interfaz de usuario para el inicio de sesión de la aplicación PLUTTER Innovation. El fondo es verde. En el centro superior hay un logo con una 'P' azul y el texto 'PLUTTER Innovation'. Debajo del logo, el título 'Comprobar Autenticación de Usuario' aparece en un recuadro verde. Hay dos campos de entrada blancos: 'Correo Electrónico' y 'Contraseña'. Debajo de ellos está un botón azul con el texto 'Ingresar'. En la parte inferior, hay un enlace azul que dice '¿Olvidaste tu contraseña?'.

Figura 59- Aplicación: Página de Log In

Esta imagen es una captura de la misma interfaz de inicio de sesión que la Figura 59, pero mostrando un estado de error. Los campos 'Correo Electrónico' y 'Contraseña' ahora contienen el texto '06106915619@uma.es' y '.....' respectivamente. El botón 'Ingresar' sigue siendo azul. Sin embargo, debajo del botón, un mensaje de error en color rojo dice 'Correo o contraseña incorrectos'. El enlace '¿Olvidaste tu contraseña?' sigue presente en la parte inferior.

Figura 60- Aplicación: Ejemplo para credenciales incorrectas

12.2 Página de restablecimiento de contraseña

Como se ha explicado en ocasiones anteriores, la aplicación tiene habilitada una página de recuperación de credenciales extraviadas. Tras pulsar '¿Olvidaste tu contraseña?' en la página de autenticación, se abre la siguiente interfaz:

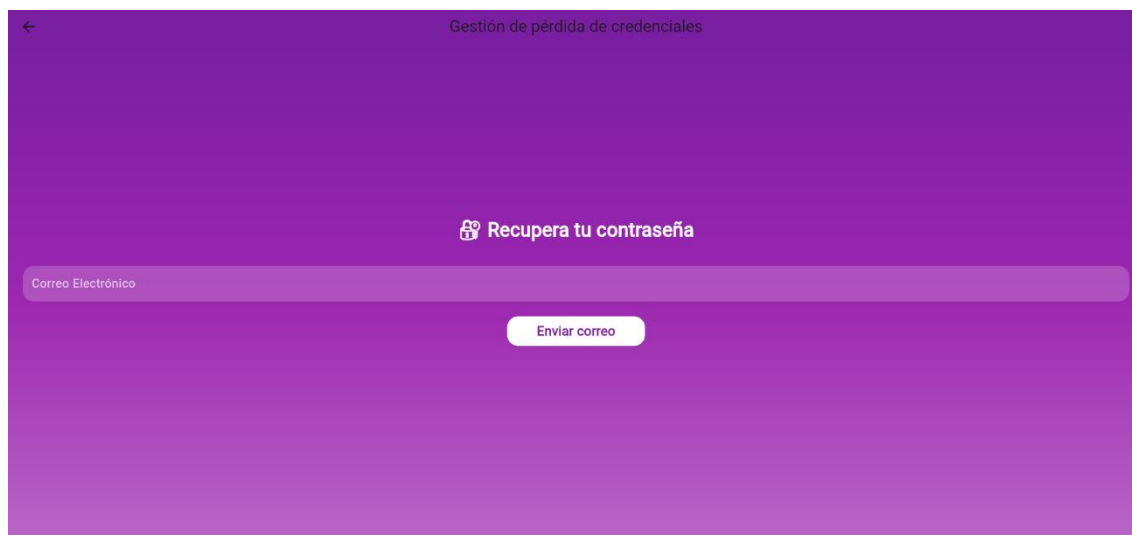


Figura 61- Aplicación: Página de restablecimiento de contraseña

Dado que debe existir un correo asociado, en esta pestaña deberás introducir tu email como usuario y pulsar el botón habilitado para enviar un correo y continuar con los pasos para generar una nueva contraseña. En otro caso, podrás volver a la pantalla de inicio de sesión de nuevo con el botón presente arriba izquierda de la pantalla:



Figura 62- Aplicación: Botón de retroceso a la página de autenticación

Vamos a realizar una prueba de recuperación, en la que seguiremos los pasos para cambiar nuestra contraseña asociada. Tras introducir la dirección email y pulsar el botón de 'Enviar correo', nos aparece un mensaje de éxito:

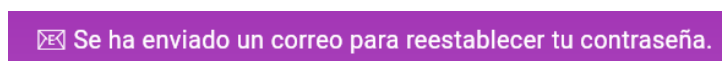


Figura 63- Aplicación: Mensaje de correo enviado

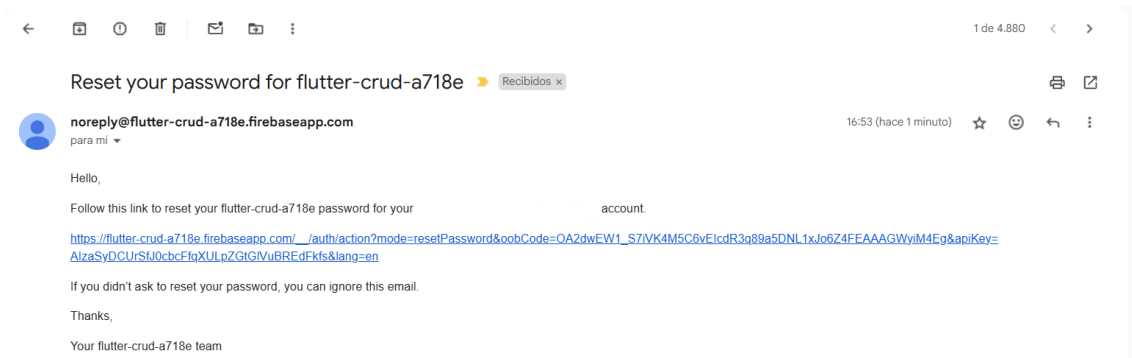


Figura 64- Aplicación: Correo para la recuperación de contraseña

Como podemos apreciar, en mi bandeja de entrada se ha recibido el email correspondiente, en el que se nos ofrece un enlace para el restablecimiento. Este mensaje se genera automáticamente por parte del servicio de autenticación ofrecido por Firebase. Si pulsamos en el link:

Reset your password

for **example@gmail.com**

New password

SAVE

Figura 65- Aplicación: Interfaz de reset

Tras haber acabado de seleccionar la nueva contraseña, accionamos el botón 'Save' y podemos volver a la pantalla de Log In para probarla y finalmente acceder a la aplicación.

12.3 Home Page

En este apartado se presenta el inicio de la aplicación. Es la ventana principal que nos encontramos al acceder correctamente al sistema. En esta imagen se pueden ver mensajes de bienvenida a la aplicación, imágenes decorativas, enlaces de visitas con webs alternativas para indagar más información y una barra inferior de navegación que servirá para alternar entre las diferentes ventanas y funciones que contiene la app.

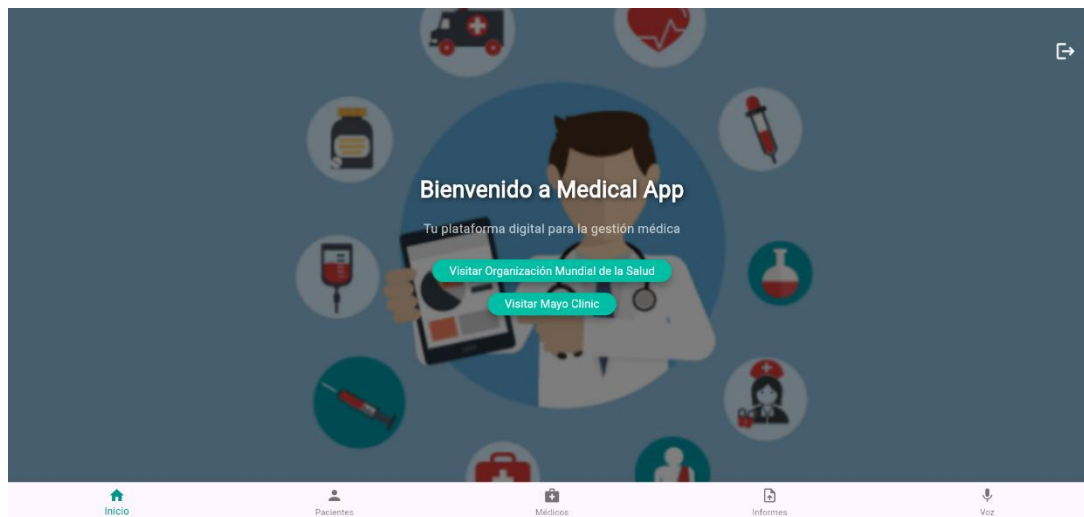


Figura 66- Aplicación: Home Page

Si pulsamos el primer enlace existente, automáticamente se crea una redirección hacia una web distinta. Veríamos algo como esto:



Figura 67- Aplicación: Web alternativa

Otro elemento presente en la interfaz sería el ícono en la parte superior de la ventana con un diseño de una puerta con una flecha. En cualquier momento, el usuario que esté usando la aplicación, podrá dirigirse a la 'Home Page' para cerrar su sesión, que es para lo que está diseñado este botón.



Figura 68- Aplicación: Botón para cierre de sesión

Tras pulsarlo, como en cualquier cierre de sesión habitual, se nos preguntará si realmente queremos hacer 'Log Out' o nos hemos equivocado al pulsarlo. Este mensaje de confirmación aparecerá en mitad de la pantalla:

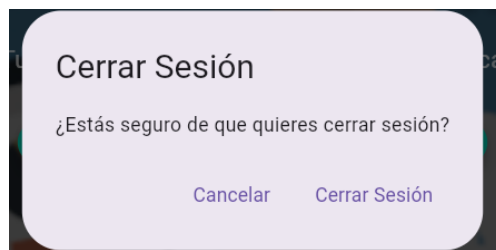


Figura 69- Aplicación: Mensaje de cierre de sesión

12.4 Páginas de gestión de personas

Este apartado englobará tanto la gestión de pacientes como la de médicos. Debido a que su implementación es la misma, se tratarán de forma conjunta, aunque mostrando las breves diferencias que puedan existir. Estas páginas, se corresponden con los iconos de 'Pacientes' y 'Médicos' mostrados en la 'Home Page'.

De manera clara y ordenada, se representa un listado de las personas existentes en la base de datos del proyecto, que han podido ser introducidos manualmente en esta, o a través de la propia aplicación como se explicará más adelante.

Podemos ver una serie de cartas propias para cada individuo existente con una serie de atributos que proporcionan detalles sobre estas personas. Observamos 3 Pacientes y 2 Médicos actualmente, pero podríamos añadir tantos como quisiéramos.

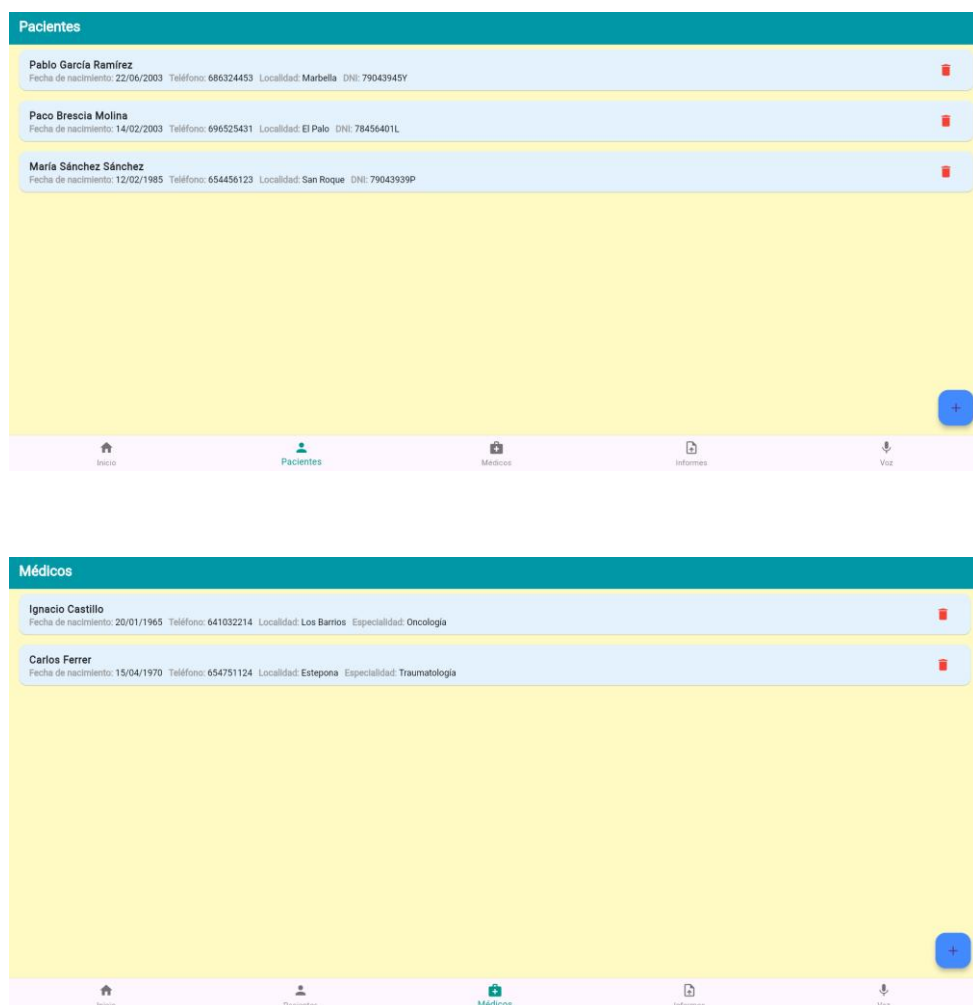


Figura 70- Aplicación: Ventanas de gestión de pacientes y médicos

Aunque parezcan estáticas a simple vista, estas ventanas tienen varios **elementos interactivos** en ellas. Esto significa que, haciendo clic en dichos elementos, se activan funciones aparentemente ocultas, pero que tienen un uso importante para la gestión de dichos pacientes. Empecemos de más a menos visible comentando cada uno de ellos:

1. En primer lugar, en ambas pestañas aparece un botón con el símbolo '+' en la zona inferior derecha. Si pulsamos este botón, se abre una nueva pestaña de creación de personas y, dependiendo de si estábamos previamente en la página de 'Pacientes' o en la de 'Médicos', se habilitarán las opciones para rellenar los atributos del correspondiente.

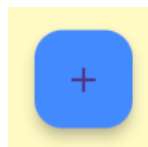


Figura 71- Aplicación: Botón de creación de personas

Las imágenes muestran dos interfaces de usuario para la creación de registros. La primera, titulada 'Agregar Paciente', contiene campos de entrada para 'Nombre', 'DNI', 'Fecha de Nacimiento (DD/MM/AAAA)', 'Teléfono' y 'Localidad', con un botón 'Guardar' debajo. La segunda, titulada 'Agregar Médico', contiene campos para 'Nombre', 'Especialidad', 'Fecha de Nacimiento (DD/MM/AAAA)', 'Teléfono' y 'Localidad', también con un botón 'Guardar' debajo. Ambas interfaces tienen una barra superior azul con un icono de retroceso y el título de la acción.

Figura 72- Aplicación: Ventanas de creación de personas

Una vez añadidos los datos, si pulsamos en 'Guardar' una nueva persona quedará añadida en la colección correspondiente de la base de datos y quedará reflejada en la interfaz de usuario.

2. Consecutivamente, al final de cada línea de las listas que representan las diferentes modalidades de persona, aparece un icono de una papelera en color rojo. Como es posible intuir, este icono se utiliza como un acceso rápido para eliminar registros. Si hacemos clic sobre la papelera correspondiente a la línea del objetivo que queremos eliminar, este directamente desaparece de la IU y queda borrado de su respectiva colección en la base de datos.



Figura 73- Aplicación: Icono de eliminación de personas

3. Para finalizar, en el tercer lugar tenemos al elemento más escondido, pero no menos útil, y es que estamos hablando de la edición de los campos de las personas registradas. Esto se consigue pulsando sobre el recuadro que compone cada nombre y automáticamente se despliega la pestaña de edición correspondiente.

En este caso, ocurre de forma idéntica al proceso de creación. El ejemplo de ejecución para la edición de los campos de un paciente sería el siguiente:

Paco Brescia Molina

Fecha de nacimiento: 14/02/2003 Teléfono: 696525431 Localidad: El Palo DNI: 78456401L

Figura 74- Aplicación: Inicio de la edición de un paciente

A screenshot of a mobile application interface titled "Editar Paciente". It features a blue header with a back arrow and the title. Below the header, there are six input fields with labels and pre-filled values: "Nombre" (Paco Brescia Molina), "Fecha de Nacimiento" (12/01/2000), "Teléfono" (678654124), "Localidad" (Fuengirola), and "DNI" (77642121J). At the bottom, there is a blue button labeled "Actualizar".

A screenshot of the same "Editar Paciente" form, but with updated data. The input fields now contain: "Nombre" (Jesús Martínez Morales), "Fecha de Nacimiento" (14/02/1980), "Teléfono" (621123354), "Localidad" (El Palo), and "DNI" (77548787B). The "Actualizar" button remains at the bottom.

Figura 76- Aplicación: Cambios aplicados en el paciente

Jesús Martínez Morales

Fecha de nacimiento: 14/02/2003 Teléfono: 621123354 Localidad: El Palo DNI: 77548787B

Figura 75- Aplicación: Final de la edición de un paciente

12.5 Página de análisis de informes médicos

Como se ha recalcado numerosas veces a lo largo de esta memoria, sabemos con certeza cuál es esta funcionalidad de la aplicación, pero veamos con detalle los pasos que conlleva hacer el proceso en primera persona desde la interfaz de usuario de la aplicación.

Al igual que para las páginas anteriores, será necesario hacer clic en el icono correspondiente a la sección de 'Informes' para navegar hasta la pestaña de subida. Esto es lo que aparece al situarnos en dicha ventana:

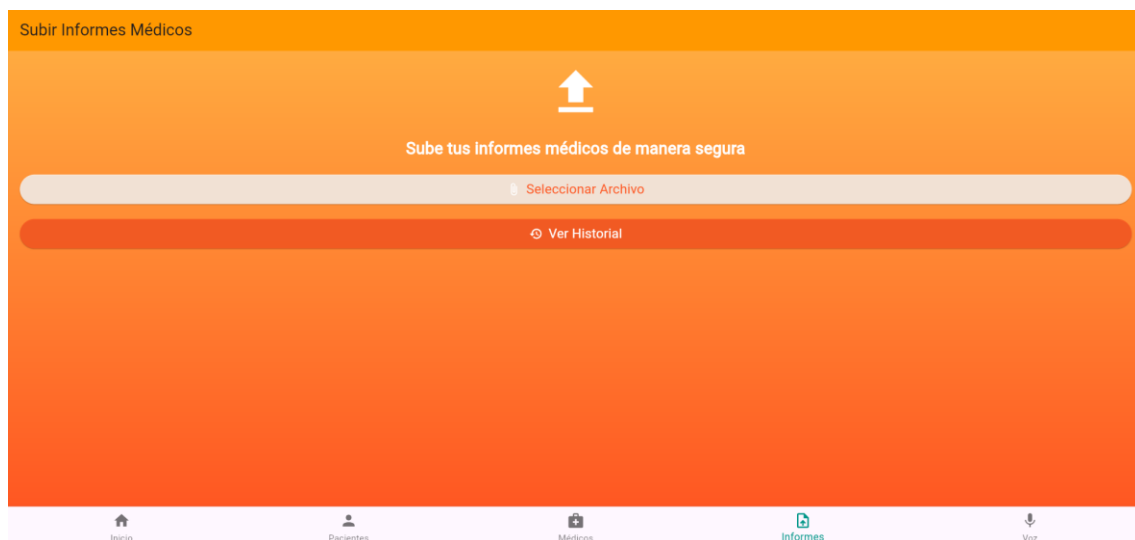


Figura 76- Aplicación: Ventana de Informes Médicos

Como se puede ver, el icono correspondiente a la página en la barra inferior de navegación está resaltado en un tono verde, lo que indica que es la pestaña sobre la que estamos situados. Una vez aquí, se presentan unos colores llamativos de fondo, una serie de títulos indicativos, un símbolo que indica carga y un botón para seleccionar el archivo que deseamos analizar.

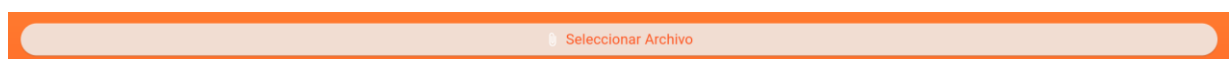


Figura 77- Aplicación: Botón para la carga de archivos

Este botón interactivo será el encargado de abrir el explorador de archivos de nuestro dispositivo personal, permitiendo seleccionar los correspondientes con el formato deseado.

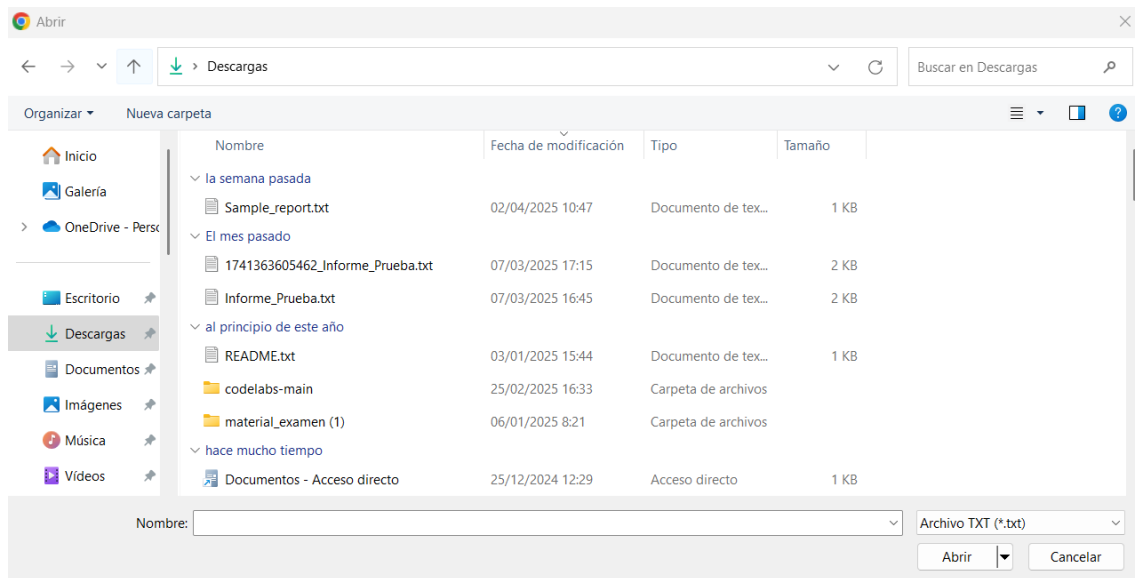


Figura 78- Explorador de archivos

Para hacer un test, seleccionamos el archivo con el nombre de 'Sample_report' que contiene el siguiente texto: "Paciente con fiebre, mocos y dolor de garganta".

Archivo seleccionado: Sample_report.txt

Figura 79- Aplicación: Mensaje de éxito en la carga de archivo

Como se puede apreciar en la Figura previa, la app lanza un mensaje hacia la interfaz que indica el archivo que ha sido seleccionado para ser analizado.

De forma más técnica, también se imprimen ciertos comentarios indicativos en la terminal de ejecución, para comprobar que todos los pasos se están completando con certeza:

```
A Dart VM Service on Chrome is available at: http://127.0.0.1:56929/p5ug6RbHm8=
The Flutter DevTools debugger and profiler on Chrome is available at: http://127.0.0.1:9102?uri=http://127.0.0.1:56929/p5ug6RbHm8=
Archivo subido en:
https://firebasestorage.googleapis.com/v0/b/flutter-crud-a718e.firebaseio.com/o/reports%2F1744037475234_Sample_report.txt?alt=media&token=5863e081-f507-4e08-9b23-360e0942b50e
Texto extraído antes de enviar a IA: Paciente con fiebre, mocos y dolor de garganta.
Enviando texto a Firebase Functions...
```

Figura 80- Aplicación: Mensajes indicativos por la terminal

Los primeros mensajes son predeterminados y se lanzan al correr la aplicación, pero tras la carga del archivo podemos comprobar cómo se indica la referencia de almacenamiento, se extrae el texto del archivo de forma correcta, y este es enviado a Firebase Functions para que conecte con el modelo alojado de inteligencia artificial alojado en HuggingFace.

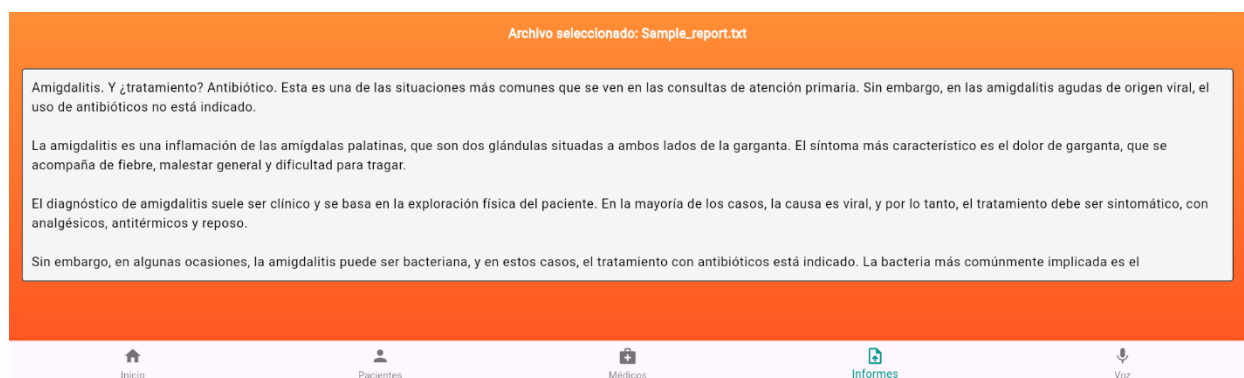


Figura 81- Aplicación: Diagnóstico mostrado en la IU

Dado que el diagnóstico es extenso, se ha habilitado un recuadro de visualización integrado de en la interfaz, con formato de 'scroll', para poder leer con claridad y sencillez las indicaciones que nos proporciona la inteligencia artificial tras analizar nuestro prospecto.

Además de esto, la interfaz presenta un botón interactivo adicional para la revisión del historial de los informes almacenados con anterioridad:



Figura 82- Aplicación: Botón para visualizar historial de archivos

Tras pulsar este botón, aparecen por pantalla una serie de 'cards' similares a los que representan las personas registradas en la base de datos. Estas franjas muestran los archivos que han sido subidos con anterioridad para tener un control de estos y un acceso rápido para consultas.



Figura 83- Aplicación: Historial de archivos

El formato es claro y visual para facilitar el manejo y entendimiento del usuario, los 'cards' se despliegan de manera ordenada y coherente a lo largo de la pantalla. En caso de muchos archivos, la interfaz ofrece un formato de scroll para evitar el desbordamiento.

Adicionalmente, estos archivos poseen funcionalidad integrada de eliminación, a través del icono de papelera como en página anteriores, y visualización de contenido tras hacer clic encima de ellos.

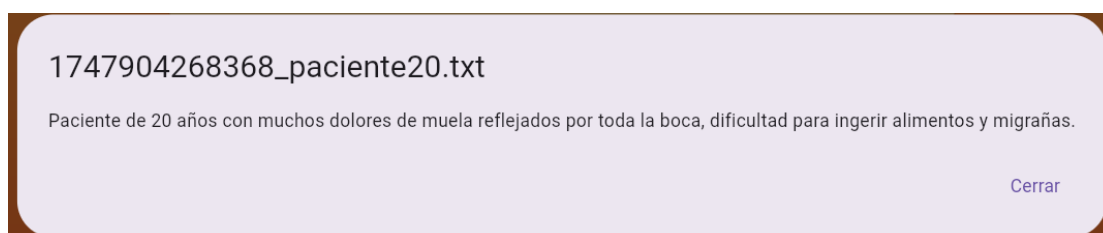


Figura 84- Aplicación: Contenido de archivos

La Figura anterior enseña el despliegue del contenido que se almacenó en dicho archivo. Nótese que el nombre del archivo de texto aparece con la marca única de tiempo añadida por Firebase Storage para evitar duplicados, tal y como se almacenan en esta herramienta. Abajo derecha, pulsando el botón de 'Cerrar', o en otro lugar de la pantalla, volveríamos a la sección anterior.

Por último, comentar que este historial se puede mostrar o no a conveniencia del usuario a través del botón de interacción, que en este caso tiene un título distinto:

Figura 85- Aplicación: Botón para visualizar ocultar de archivos

12.6 Página de reconocimiento de voz y análisis

Esta es la última sección disponible de la aplicación, donde se mezclan varias funcionalidades y hay más de una ventana involucrada. Siguiendo el esquema de los apartados anteriores, se hará un resumen visual del proceso de funcionamiento de las herramientas que ofrece esta última sección.

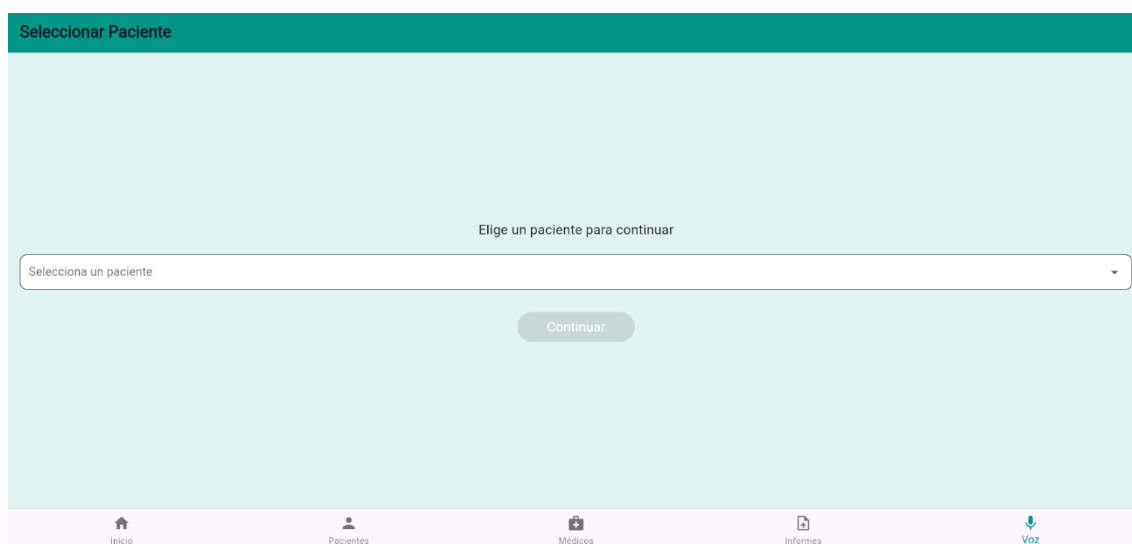


Figura 86- Aplicación: Ventana de selección de pacientes

En el quinto icono de la barra de navegación se encuentra la distribución de interfaz ilustrada en la captura anterior. Como se explicaba en capítulos anteriores, este servicio ofrece una atención personalizada a cada paciente, por lo que es **necesario** una selección previa de los mismos.

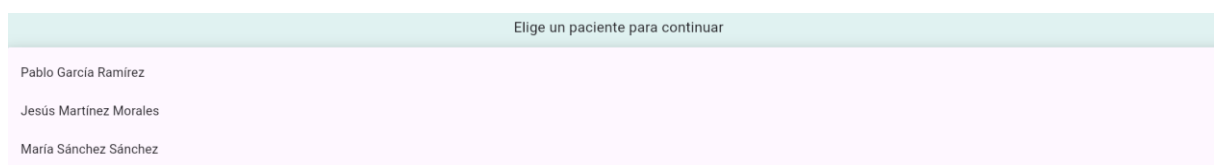


Figura 87- Aplicación: Desplegable de selección de pacientes

Si pulsamos el botón desplegable de la Figura 83, se abrirá el panel previo que muestra, si prestamos atención, los mismos pacientes que estaban en la lista de la ventana 'Pacientes', y que corresponden a los registrados en la base de datos.

Al seleccionar cualquiera de ellos para el que queramos personalizar el servicio, se activa el botón de 'Continuar'.

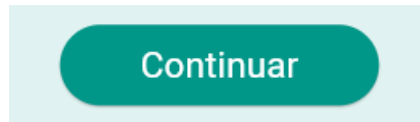


Figura 88- Aplicación: Botón de continuación

Este botón, dará paso a una nueva ventana disponible que da nombre al título de este apartado.



Figura 89- Aplicación: Ventana de diagnóstico personalizado

En esta nueva imagen, se presentan una interfaz limpia, clara y con diferentes funciones. Como podemos observar, el título de la cabecera de página está personalizado para el paciente que queremos tratar. Una vez más, esta pantalla es interactiva y ofrece varios elementos de funcionamiento.

Esta pestaña de la aplicación permite mantener una conversación privada con la inteligencia artificial, de forma que el médico o el administrador pueda reiterar consultas acerca de un paciente concreto o hacer nuevas peticiones en este.

Tiene un formato de chat convencional con formato de 'scroll' en caso de conversaciones más duraderas. En esta imagen anterior, se ha aprovechado la oportunidad de incluir un mensaje de error, en el que el modelo no recibió correctamente la solicitud enviada y no se recibió una respuesta certera.



Figura 90- Aplicación: Botón de retroceso en la selección de pacientes

En cualquier momento es posible retroceder hacia las pestañas comentadas anteriormente, si necesitamos navegar hacia otra parte de la app, o simplemente seleccionar un nuevo paciente para dedicar los servicios.

Si analizamos el resto de la interfaz, tenemos un botón de micrófono, que pedirá permisos al navegador para empezar a usar los servicios de voz.

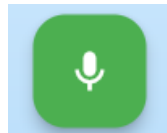


Figura 91- Aplicación: Botón de acción del micrófono

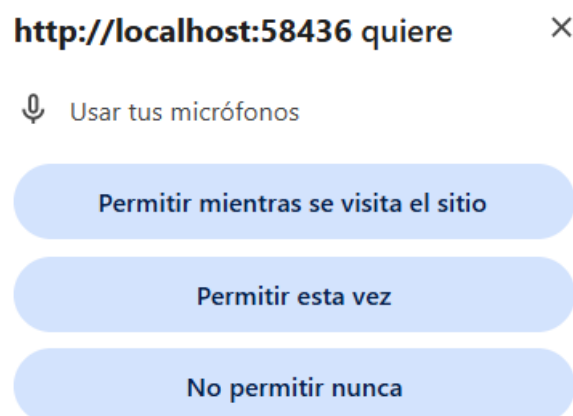


Figura 92- Aplicación: Permisos de uso de micrófono

Una vez permitido su uso, se muestra un cambio de forma del botón previo, indicando así su dinamismo:



Figura 93- Aplicación: Botón de STOP del micrófono

Si de nuevo pulsamos el botón, se cortaría la grabación actual y aparecería en el recuadro indicado el texto reconocido. Si no se habla durante la grabación, el texto se mantiene nulo. Vamos a notificar la misma dolencia que causó el error:

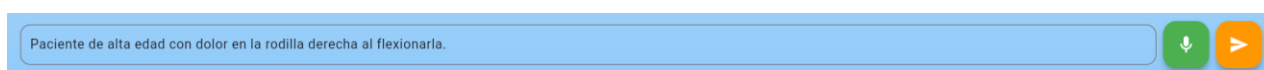


Figura 94- Aplicación: Transcripción de voz exitosa

En este caso se ha hecho una prueba para unos dolores de rodilla comunes en deportistas o personas de alta edad. El texto reconocido a través de voz ha sido correcto y he añadido mayúsculas y signos de puntuación manualmente gracias a que el recuadro permite su edición.

Para terminar con el proceso, lo único restante sería pulsar el otro botón consecutivo al del micrófono, que enviaría el texto visible en el recuadro hacia la IA para obtener el diagnóstico resultante siguiendo el proceso de la ventana de carga de informes, pero sin necesidad de tener que extraer el texto de ningún documento.



Figura 95- Aplicación: Botón de envío de texto médico hacia IA

Una vez enviado, los pasos de ejecución serían muy similares a los de análisis de informes.

```

Estado: listening
Estado: notListening
Estado: done
Enviando texto a Firebase Functions...

```

Figura 96- Aplicación: Terminal de ejecución del reconocimiento de voz

En el transcurso de la grabación, la terminal imprime mensajes informativos del estado del reconocimiento de voz, y cuando se pulsa el botón de envío aparece el mensaje explicativo que veíamos en apartados anteriores.

The screenshot shows a mobile application interface for a medical chat. At the top, a blue header bar contains a back arrow and the text "Diagnóstico de Pablo García Ramírez". Below the header, a light blue banner displays the patient's condition: "Paciente de alta edad con dolor en la rodilla derecha al flexionarla." A yellow error message at the top left states: "Error al procesar el informe. Por favor, inténtalo de nuevo más tarde." The main content area has a light gray background and lists four options: A) Fractura de rótula, B) Artrosis de rodilla, C) Tendinitis rotuliana, and D) Tendinitis de cuádriceps. Below the list, it shows the selected answer: "Respuesta: B) Artrosis de rodilla". This is followed by a section titled "## Artrosis de rodilla" with several paragraphs of text explaining the condition, its symptoms, and treatment options. At the bottom, there is a text input field with the placeholder "Escribe tu mensaje o usa el micrófono..." and two buttons: a green microphone icon and an orange send button with a right arrow.

Figura 97- Aplicación: Respuesta aplicada al chat médico

El mensaje se envía con éxito y se recibe una respuesta sobre una posible enfermedad potencial que podría portar el paciente. Esto queda reflejado en la conversación.

Como bien sabemos, una vez terminado y completado el proceso exitosamente, se procede a guardar un registro de esta acción en la base de datos con una serie de atributos propios del documento.

🏠 > patients > Ba9ZyI0ryngzPYxGeyLC > messages Más funciones en Google Cloud		
patients + Agregar documento Ba9ZyI0ryngzPYxGeyLC H0jFt287hUgHdpBhvTC8 z0hsM8zgb0ySyUiH1Lj5	Ba9ZyI0ryngzPYxGeyLC + Iniciar colección messages + Agregar campo birthdate: "22/06/2003" dni: "79043945Y" location: "Marbella" name: "Pablo García Ramírez" phone: "686324453"	messages + Agregar documento 6WsR2HdGRh3sAkeFIVf1 dLhpmzS6WxtY7bJjW1Dy j2kit9LcauTZPr34RayA wFZ84jvMyZVMgV4HZOWh

Figura 98- Base de datos: Colecciones divididas en los pacientes con historial médico

wFZ84jvMyZVMgV4HZOWh + Iniciar colección + Agregar campo content: "Paciente de alta edad con dolor en la rodilla derecha al flexionarla." sender: "usuario" timestamp: 14 de mayo de 2025, 11:42:33 a.m. UTC+2
--

Figura 99- Base de datos: Registro de la conversación almacenado

En esta última fotografía se muestra cómo los mensajes son archivados en la base de datos tras ser enviados por el chat personalizado. De esta forma, si volvemos atrás en la aplicación, podremos seleccionar los pacientes a ser tratados y las conversaciones seguirán presentes al momento de entrar en ellas.

Como se puede observar, se han agregado los campos de contenido, emisor y fecha de los mensajes.

Con esta pestaña finaliza la documentación de la aplicación, ya que es el último apartado presente en ella.

Capítulo 13. Referencias bibliográficas

13.1 Fuentes utilizadas, estándares y documentación técnica.

A continuación, se enumeran las principales fuentes consultadas para el desarrollo del presente trabajo, clasificadas por temática:

Inteligencia Artificial y su aplicación en medicina

1. ¿Qué es la inteligencia artificial en medicina? Recuperado de: (<https://www.ibm.com/es-es/topics/artificial-intelligence-medicine>)
2. La inteligencia artificial en el reconocimiento de voz. Recuperado de: (<https://www.invoxmedical.com/blog/la-inteligencia-artificial-en-el-reconocimiento-de-voz-32951>)

Aplicaciones similares existentes

3. Desarrollo acelerado de fármacos, Insilico Medicine. Recuperado de: (<https://insilico.com/>)
4. Análisis de imágenes médicas, Lunit Insight CXR. Recuperado de: (<https://www.lunit.io/en/products/cxr>)

Herramientas de análisis de texto con IA

5. Las mejores herramientas de análisis de texto con IA. Recuperado de: (<https://www.edrawsoft.com/es/article/top-ai-text-analysis-tools.html>)

Documentación de los frameworks y tecnologías utilizadas

6. Documentación oficial de Flutter. Recuperado de: (<https://flutter.dev/>)
7. Plataforma oficial de Firebase. Recuperado de: (<https://firebase.google.com/?hl=es-419>)
8. Página oficial de Oracle. Recuperado de: (<https://www.oracle.com/>)

Otras tecnologías involucradas

9. NodeJS. Recuperado de: (<https://nodejs.org/>)
10. JavaScript. Recuperado de: (<https://developer.mozilla.org/en-US/docs/Web/JavaScript>)
11. Postman. Recuperado de: (<https://www.postman.com/>)

12. HTTP/HTTPS. Recuperado de:

(<https://developer.mozilla.org/en-US/docs/Web/HTTP>)

13. JSON. Recuperado de: (<https://www.json.org/>)

14. Visual Studio Code. Recuperado de: (<https://code.visualstudio.com/>)

